

Schedule Builder

Anthony Johnson

Introduction

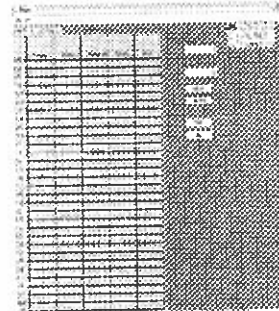
- ❖ Schedule Builder
 - About Schedule Builder
- ❖ Purpose
 - Provide a user friendly program
- ❖ Motivation
 - Learn a new language
 - Develop a program with a user interface
 - Process of developing software

Background Knowledge

- ❖ C#
 - C# Windows Forms
 - C# Syntax is similar to C++
- ❖ Microsoft Access
 - Provides good data storage
 - Capabilities of grouping information
 - Ways of Accessing Data

Design of User Interface

- ❖ Tabs
 - A tab for each of the 3 rooms
- ❖ Combo boxes
 - Hold Professor and Course names
- ❖ Consists of four buttons
 - Add
 - Delete
 - Undo
 - Clear



Database Design

- ❖ Tables
 - Professor
 - Course
 - Schedule
- ❖ Relationships
 - Professor 1 to many relationship with schedule
 - Course 1 to many relationship with schedule



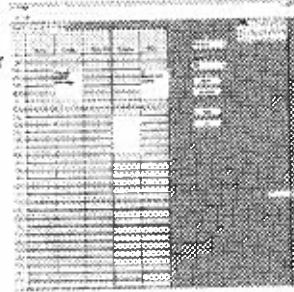
Connecting C# to Access Database

- ❖ Connection created using System.Data.OleDb Namespace with SQL
- ❖ SQL commands to read and write to database



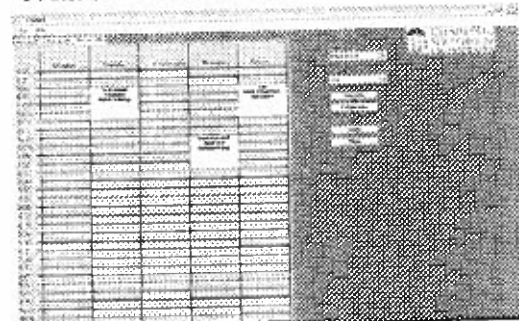
Add Button

- ❖ User Selects Room
- ❖ User Selects Time for Course
- ❖ User Selects Professor & Course
- ❖ Click Add

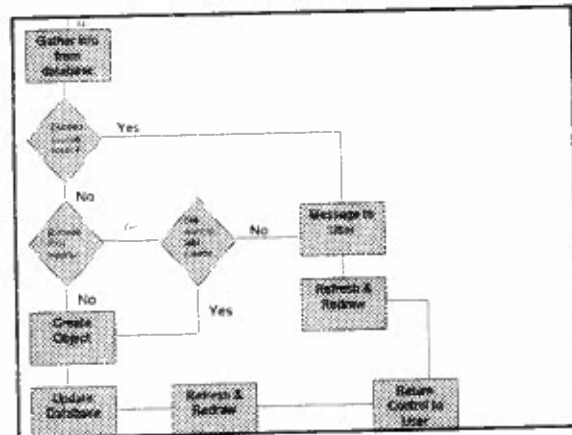
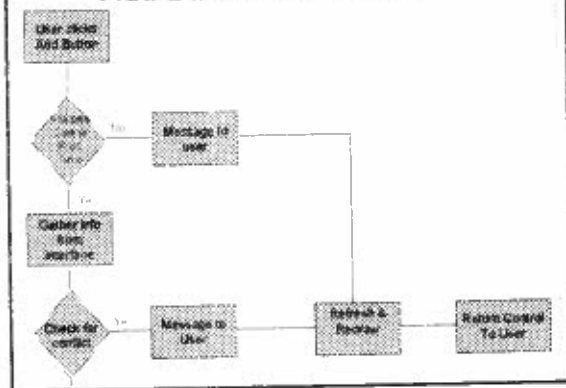


Add Button

- ❖ After add button is selected

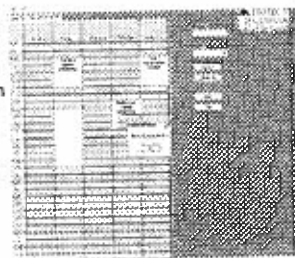


Add Button Flow Chart



Update Drawing & Re-drawing

- ❖ Update Drawing
 - When add button is clicked
 - First opening program
- ❖ Re-drawing
 - Window layover
 - Other events



Demonstration

- ❖ Scenarios
 - Professor Not selected
 - Course exceeded hours
 - Conflict in schedule
 - Delete Information

Conclusion & Future Works

- ◆ Add Double buffering
- ◆ Queries
- ◆ Administrative
- ◆ Log-in feature

Achievements

- ◆ Process of developing software
- ◆ C#
- ◆ C# & Access
- ◆ Aspect of software

References

- ◆ Windows Forms Programming in C# by Chris Sells
- ◆ Head First C# by Andrew Stellman & Jennifer Greene

```

using System;
using System.Collections.Generic;
using System.Text;
using System.Drawing;

namespace srpoj
{
    public class scheduletrckr
    {
        public String Prof;
        //Course Name and Course Number and Length of Course
        public String courseName;
        public int courseNum;
        public double Amtoftime;
        //X & Y Coordinates for upper left corner of highlighted region
        public int downx;
        public int downy;
        // X & Y Coordinate for bottom right corner of highlighted region
        public int upxc;
        public int upyc;
        //start time & end time of course
        public int startTime;
        public int endTime;
        //Day of week for course and room number
        public int dayDate;
        public int roomNum;

        //Constructor that sets attributes in object
        //Used when adding a new object to the array
        public scheduletrckr(String ProfA, String Course, int CourseN, double AmtTime,
int dwx, int dwy, int upx, int upy)
        {
            Prof = ProfA;
            courseName = Course;
            courseNum = CourseN;
            Amtoftime = AmtTime;
            downx = dwx;
            downy = dwy;
            upxc = upx;
            upyc = upy;
            //Calculates start time, end time, day of week
            setAttributes2();
        }

        //Constructor that sets attributes in object
        //used when bringing in data from database
        public scheduletrckr(String ProfA, String Course, int CourseN, int sTime, int
eTime, int dDate, int room)
        {
            Prof = ProfA;
            courseName = Course;
            courseNum = CourseN;
            startTime = sTime;
            endTime = eTime;
            dayDate = dDate;
            roomNum = room;
            setAttributes();
        }

        // calculates and sets height for object
        public int getheight()
        {
            int heightfrrec = upyc - downy;
            return heightfrrec;
        }
    }
}

```

```
}
// calculates and sets X & Y coordinates for upper-left and bottom right
// of object and calculates length of course
public void setAttributes()
{
    int natx = ((dayDate - 1) * 100) + 50;
    downx = (((natx - 50) / 100)) * 100 + 50;
    int naty = ((startTime - 1) * 20) + 60;
    downy = (((naty - 60) / 20)) * 20 + 60;
    int naty2 = ((endTime - 1) * 20) + 60;
    upyc = (((naty2 - 60) / 20)) * 20 + 60 + 20;
    int natx2 = ((dayDate - 1) / 100) + 50;
    upxc = downx+100;

    Amtoftime = (((double)endTime - (double)startTime) / 2 + .5);
}

//calculates and sets start time, end time, and day of the week
public void setAttributes2()
{
    int natx = ((downx - 50)/100) *100 + 50;
    dayDate = ((natx - 50) / 100) + 1;
    int naty = ((downy - 60) / 20) * 20 + 60;
    startTime = ((naty - 60) / 20) + 1;
    int naty2 = (((upyc - 20) - 60) / 20) * 20 + 60;
    endTime = ((naty2 - 60) / 20) + 1;
}

}

}
//*****end of Scheduletrckr class*****\\
```

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
using System.Drawing.Drawing2D;
using System.Data.OleDb;

namespace srpoj
{
    public partial class Form1 : Form
    {
        //used to get x & y coordinates from mouse events
        int x, y, hx, hy;
        int Datef, Dhour, Uphour;
        String Lname, Fname;
        double hour;
        bool mouse_down = false;
        bool sized_changed = false;
        //data structure used to keep track of schedule for each room(tab)
        scheduletrckr[] schtrckrArray = new scheduletrckr[20];
        scheduletrckr[] schtrckrArray2 = new scheduletrckr[20];
        scheduletrckr[] schtrckrArray3 = new scheduletrckr[20];
        //Keep track of how many objects are in the array
        int countClass = 0;
        int countClass2 = 0;
        int countClass3 = 0;
        //x & y for mouse up event
        int bttmx, bttmy;
        //keeps track of the object that is to be deleted
        int countDelete = -1;

        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            //Setting attributes for tabcontrol
            tabControl1.Anchor = AnchorStyles.Top | AnchorStyles.Left | AnchorStyles.
            Bottom;
            tabControl1.Size = new Size(560, ClientSize.Height);
            //sets picture in right hand corner of form
            pictureBox1.Image = Image.FromFile("CSUlogo.jpg");

            //*****Connection to database*****\\
            //*****\\

            //Handles the connection to database and also queries the table needing to be
            accessed
            OleDbConnection aConnection = new OleDbConnection("Provider=Microsoft.Jet.
            OLEDB.4.0;Data Source=srproj.mdb");
            OleDbCommand aCommand = new OleDbCommand("select * from Professor",
            aConnection);
            OleDbCommand aCommand1 = new OleDbCommand("select * from Course",
            aConnection);

            //Connecting to database
            try
            {
                aConnection.Open();
            }
        }
    }
}

```

```

//create the datareader object to connect to table
OleDbDataReader aReader = aCommand.ExecuteReader();
OleDbDataReader aReader1 = aCommand1.ExecuteReader();

//Iterate through the database Professor table
while (aReader.Read())
{
    //adds professor names to combo box
    Lname = aReader.GetString(1);
    comboBox1.Items.Add(Lname);
    Fname = aReader.GetString(2);
    hour = aReader.GetDouble(3);
}

//close the reader
aReader.Close();

//Reading Course ID from Course table
while (aReader1.Read())
{
    //adds course number to second combo box
    comboBox2.Items.Add(aReader1.GetInt32(0));
}
//close reader
aReader1.Close();

//close the connection
aConnection.Close();
}

//Some usual exception handling
catch (OleDbException ex)
{
    Console.WriteLine("Error: {0}", ex.Errors[0].Message);
}

/*Connecting to database to access schedule table and also
 * reads in rows from schedule table into the corresponding
 * array that is associated with each room*/
OleDbCommand aCommand2 = new OleDbCommand("select * from Schedule",
aConnection);

try
{
    aConnection.Open();

    //create the datareader object to connect to table
    OleDbDataReader aReader3 = aCommand2.ExecuteReader();

    //Iterate through the database Schedule table
    while (aReader3.Read())
    {
        int room = aReader3.GetInt32(6);
        // decides which array to put it in by testing the
        //room number.
        if (room == 203)
        {
            //creating object for tab that is associated with room 203
            schtrckrArray[countClass] = new scheduletrckr(aReader3.GetString(
(2), aReader3.GetString(1),
aReader3.GetInt32(0), aReader3.GetInt32(3), aReader3.GetInt32(
(4), aReader3.GetInt32(5),
aReader3.GetInt32(6));

```

```

        countClass++;
    }
    else if (room == 207)
    {
        //creating object for tab that is associated with room 207
        schtrckrArray2[countClass2] = new scheduletrckr(aReader3.
GetString(2), aReader3.GetString(1),
        aReader3.GetInt32(0), aReader3.GetInt32(3), aReader3.GetInt32(
(4), aReader3.GetInt32(5),
        aReader3.GetInt32(6));

        countClass2++;
    }
    else
    {
        //creating object for tab that is associated with room 208
        schtrckrArray3[countClass3] = new scheduletrckr(aReader3.
GetString(2), aReader3.GetString(1),
        aReader3.GetInt32(0), aReader3.GetInt32(3), aReader3.
GetInt32(4), aReader3.GetInt32(5),
        aReader3.GetInt32(6));

        countClass3++;
    }
    //close the reader
    aReader3.Close();
}

catch (OleDbException ex)
{
    aConnection.Close();

    Console.WriteLine("Error: {0}", ex.Errors[0].Message);
}

}

private void Form1_Paint(object sender, PaintEventArgs e)
{
    //ResizeRedraw = true;
    //if (sized_changed)
    //{
        //    if (WindowState == FormWindowState.Maximized)
        //    {
        //        displaySchedule();
        //    }

        //    if (WindowState == FormWindowState.Normal)
        //    {
        //        displaySchedule();
        //    }
    //}
}

private void exitToolStripMenuItem_Click(object sender, EventArgs e)
{
    Close();
}

private void saveToolStripMenuItem_Click(object sender, EventArgs e)
{

```



```

    }

    /*******TabPage1 Objects*****
private void tabPage1_MouseDown(object sender, MouseEventArgs e)
{
    //calculating start time
    Dhour = (int)((e.Y - 60) / 20) + 1;
    //calculating day of week
    Datef = (int)((e.X - 50) / 100) + 1;
    //calculates points for upper-left corner of highlight
    hx = ((int)((e.X - 50) / 100)) * 100 + 50;
    hy = ((int)((e.Y - 60) / 20)) * 20 + 60;
    //boolean that notifies when mouse is down
    mouse_down = true;
}

private void tabPage1_MouseMove(object sender, MouseEventArgs e)
{
    //x & y coordinates for mouse move event
    x = e.X;
    y = e.Y;
    //When mouse down is true, graphics will fill the specified area
    if (mouse_down)
    {
        //fills specified area on tab for room 203
        using (Graphics g = tabPage1.CreateGraphics())
        {
            if ((x >= 50) && (y >= 60))
                g.FillRectangle(Brushes.LightGoldenrodYellow, ((int)((x - 50) /
100)) * 100 + 50, ((int)((y - 60) / 20)) * 20 + 60, 100, 20);
        }
    }
}

private void tabPage1_Paint(object sender, PaintEventArgs e)
{
    //redraws tab page when it is resized by user
    ResizeRedraw = true;

    //Draws design for User Interface
    Graphics g = e.Graphics;
    Pen p = new Pen(Color.Black, 2);
    int i, j;
    i = j = 60;

    //draws horizontal lines
    while (i < 670)
    {
        Point pt1 = new Point(40, i);
        Point pt2 = new Point((550), j);
        g.DrawLine(p, pt1, pt2);
        i += 20;
        j += 20;
    }

    Pen p1 = new Pen(Color.Blue, 2);
    int k, l;

```

```

    k = 1 = 50;

    //draws vertical lines
    while (k < 560)
    {
        Point pt3 = new Point(k, 0);
        Point pt4 = new Point(1, ClientSize.Height);
        g.DrawLine(p1, pt3, pt4);
        k += 100;
        l += 100;
    }

    Font font = new Font("Arial", 10);

    //Drawing days of the week
    g.DrawString("Monday", font, Brushes.Black, 75, 40);
    g.DrawString("Tuesday", font, Brushes.Black, 175, 40);
    g.DrawString("Wednesday", font, Brushes.Black, 275, 40);
    g.DrawString("Thursday", font, Brushes.Black, 375, 40);
    g.DrawString("Friday", font, Brushes.Black, 475, 40);

    int mini = 0;
    int houri = 8;
    int position = 52;

    //Drawing Hours of the Day
    while (position < 660)
    {
        g.DrawString(houri.ToString() + ":" + mini.ToString() + "0", font,
Brushes.Black, 0, position);
        position += 20;
        mini += 3;
        if (mini == 6)
        {
            houri++;
            mini = 0;
        }
        if (houri == 13)
            houri = 1;

    }

    //Locates what position mouse is in and fills time & day
    int hour_index, date_index;
    date_index = (int)((x - 50) / 100) + 1;
    hour_index = (int)((y - 60) / 20) + 1;

    //Shows position of mouse
    //g.DrawString(hour_index.ToString(), font, Brushes.Black, 200,
ClientRectangle.Height - 220);
    //g.DrawString(date_index.ToString(), font, Brushes.Black, 300,
ClientRectangle.Height - 220);
    //g.DrawString(x.ToString(), font, Brushes.Black, 200, ClientRectangle.Height
- 200);
    //g.DrawString(y.ToString(), font, Brushes.Black, 300, ClientRectangle.Height
- 200);

    }

    private void tabPage1_MouseUp(object sender, MouseEventArgs e)

```

```

{
    //calculates end time
    Uphour = (int)((e.Y - 60) / 20) + 1;
    //calculates right hand corner of object
    bttmx = ((int)((e.X - 50) / 100)) * 100 + 50 + 100;
    bttmy = ((int)((e.Y - 60) / 20)) * 20 + 60 + 20;
    //boolean that is false when mouse up event is fired
    mouse_down = false;
}
/*Add button that checks data to make sure it is valid. Once
 * data is verified it then checks for and discrepancies between
 * course allowed hours, professor allowed hours and course
 * conflicts for professor. If all checks are good then it will
 * add a new object to the array according to the corresponding room
 * then updates the course, professor, and schedule table
 */

private void button1_Click(object sender, EventArgs e)
{
    //validates professor
    if (comboBox1.SelectedItem == null)
    {
        MessageBox.Show("Please Choose a Professor", "Professor Blank");
    }
    else if (comboBox2.SelectedItem == null) //validates course
    {
        MessageBox.Show("Please Choose a Course", "Course Blank");
    }
    else if ((hx < 50) && (hy < 60)) //validates that the highlight has taken
    place
    {
        MessageBox.Show("Please Choose a Time Slot", "Time Slot Blank");
    }
    else //if professor, course, and timeslot is valid
    {
        //retrieves information from combo boxes
        String Pname = comboBox1.SelectedItem.ToString();
        int CourseNum = (int)comboBox2.SelectedItem;
        //boolean to check if course conflicts
        bool check;
        //retrieves name of course from database
        String CourseNam = null;
        //prepares connection to database and query to access table
        OleDbConnection aConnection1 = new OleDbConnection("Provider=Microsoft.
Jet.OLEDB.4.0;Data Source=srproj.mdb");
        OleDbCommand getCoursel = new OleDbCommand("Select * From Course Where
CourseID = ?", aConnection1);
        getCoursel.Parameters.Add(new OleDbParameter("CourseID", CourseNum));

        try
        {
            aConnection1.Open();
            OleDbDataReader crseReader1 = getCoursel.ExecuteReader();
            while (crseReader1.Read())
            {
                //retrieves name of course
                CourseNam = crseReader1.GetString(1);
            }
            crseReader1.Close();
        }
        catch (OleDbException ex)
        {
            Console.WriteLine("Error: {0}", ex.Errors[0].Message);
        }
        finally

```

```

        {
            aConnection1.Close();
        }

        //*****Checking if professor has any conflicts in schedule*****✓
        *****//
        if (tabControl1.SelectedTab == tabPage2)
        {
            check = checkSchedule(schtrckrArray, Pname, Datef, Dhour, countClass, ✓
CourseNam) ||
            checkSchedule(schtrckrArray3, Pname, Datef, Dhour, countClass3, ✓
CourseNam);
        }
        else if (tabControl1.SelectedTab == tabPage3)
        {
            check = checkSchedule(schtrckrArray, Pname, Datef, Dhour, countClass, ✓
CourseNam) ||
            checkSchedule(schtrckrArray2, Pname, Datef, Dhour, countClass2, ✓
CourseNam);
        }
        else
        {
            check = checkSchedule(schtrckrArray2, Pname, Datef, Dhour, ✓
countClass2, CourseNam) ||
            checkSchedule(schtrckrArray3, Pname, Datef, Dhour, countClass3, ✓
CourseNam);
        }
        //checks to see if course has any conflicts with other courses
        if (check)
        {
            //if there is a conflict redraws objects on tabs
            displaySchedule();
        }
        else///pass all the checks good to process rest of data
        {
            //calculate length of course
            double Amtoftime = (((double)Uphour - (double)Dhour)/2 + .5);
            //variables for assigned hours and hours allowed
            double profAsgndHrs = 0;
            double profAmtAllowed = 0;
            //connect to database to retrieve information from professor table
            //for assigned hours and hours allowed
            OleDbConnection aConnection = new OleDbConnection("Provider=Microsoft.Jet✓
.OLEDB.4.0;Data Source=srproj.mdb");
            OleDbCommand getProf = new OleDbCommand("SELECT * FROM Professor Where ✓
LastName = ?", aConnection);
            getProf.Parameters.Add(new OleDbParameter("LastName", Pname));
            //connecting to database
            try
            {
                aConnection.Open();
                //reads data from table
                OleDbDataReader getProfReader = getProf.ExecuteReader();
                while (getProfReader.Read())
                {
                    //retrieve hours for assigned and allowed for professors
                    profAsgndHrs = getProfReader.GetDouble(4);
                    profAmtAllowed = getProfReader.GetDouble(3);
                }
                getProfReader.Close();
            }
            catch (OleDbException ex)
            {
                Console.WriteLine("Error: {0}", ex.Errors[0].Message);
            }
        }
    }
}

```

```

        finally
        {
            aConnection.Close();
        }
        //variables for assigned hours and allowed hours for course
        //and get course name from table
        double crseAsgnedHrs = 0;
        double crseAmtAllowed = 0;
        String CourseName = null;
        //handles connection for database
        OleDbCommand getCourse = new OleDbCommand("Select * From Course Where
CourseID = ?", aConnection);
        getCourse.Parameters.Add(new OleDbParameter("CourseID", CourseNum));
        //connection to database
        try
        {
            aConnection.Open();
            //reads from course table
            OleDbDataReader crseReader = getCourse.ExecuteReader();
            while (crseReader.Read())
            {
                //retrieves information from table according to query
                CourseName = crseReader.GetString(1);
                crseAmtAllowed = crseReader.GetDouble(3);
                crseAsgnedHrs = crseReader.GetDouble(4);
            }
            //close reader
            crseReader.Close();
        }
        catch (OleDbException ex)
        {
            Console.WriteLine("Error: {0}", ex.Errors[0].Message);
        }
        finally
        {
            aConnection.Close();
        }
        //if course hours is not over limit continue
        if (crseAmtAllowed >= (crseAsgnedHrs + Amtoftime))
        {
            //adding hours to assigned hours
            crseAsgnedHrs += Amtoftime;

            DialogResult result = DialogResult.Yes;
            //test if professor went over allowed hours
            if (profAmtAllowed < (profAsgnedHrs + Amtoftime))
            {
                //gives choose of adding course
                result = MessageBox.Show("You have went over Your limit. Do you
still want to "+
                    "add the course","Over Limit",MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
            }
            //if no then don't add course and display schedule
            if (result == DialogResult.No)
            {
                MessageBox.Show("Course was not added");
                displaySchedule();
            }
            else//if still want to add class or if professor hours was not over
            {
                //add hours to professor assigned hours
                profAsgnedHrs += Amtoftime;
            }
        }
    }
}

```

```

//adding information to corresponding array depending on
//which tab(room) is currently selected
if (tabControl1.SelectedTab == tabPage2)
{
    //adding course to array and drawing graphics to tabpage
    schtrckrArray2[countClass2] = new scheduletrckr(Pname,
    CourseName, CourseNum, Amtoftime, hx, hy, bttmx, bttmy);
    int recHeight = schtrckrArray2[countClass2].getheight();
    RectangleF layout = new RectangleF(schtrckrArray2
[countClass2].downx, schtrckrArray2[countClass2].downy, 100, recHeight);
    Font f = new Font("Arial", 8, FontStyle.Bold);
    using (Graphics g = tabPage2.CreateGraphics())
    {
        g.DrawString(Pname +
            "\n" + CourseName.ToString(), f, Brushes.Black,
layout);
    }
    countClass2++;
}

else if (tabControl1.SelectedTab == tabPage3)
{
    //adding course to array and drawing graphics to tabpage
    schtrckrArray3[countClass3] = new scheduletrckr(Pname,
    CourseName, CourseNum, Amtoftime, hx, hy, bttmx, bttmy);
    int recHeight = schtrckrArray3[countClass3].getheight();
    RectangleF layout = new RectangleF(schtrckrArray3
[countClass3].downx, schtrckrArray3[countClass3].downy, 100, recHeight);
    Font f = new Font("Arial", 8, FontStyle.Bold);
    using (Graphics g = tabPage3.CreateGraphics())
    {
        g.DrawString(Pname+"\n" + CourseName.ToString(), f,
Brushes.Black, layout);
    }
    countClass3++;
}

else
{
    //adding course to array and drawing graphics to tabpage
    schtrckrArray[countClass] = new scheduletrckr(Pname,
    CourseName, CourseNum, Amtoftime, hx, hy, bttmx, bttmy);
    int recHeight = schtrckrArray[countClass].getheight();
    RectangleF layout = new RectangleF(schtrckrArray[countClass].
downx, schtrckrArray[countClass].downy, 100, recHeight);
    Font f = new Font("Arial", 8, FontStyle.Bold);
    StringFormat format = new StringFormat();
    format.Alignment = StringAlignment.Center;
    using (Graphics g = tabPage1.CreateGraphics())
    {
        g.DrawString(Pname + "\n" + CourseName, f, Brushes.Black,
layout, format);
    }
    countClass++;
}

//query that updates the course table; updates the assigned hours
//for the course that was choosen
String s = "UPDATE Course SET Assignedhours = ? WHERE CourseID =
? ";

//query that updates professor table. Updates the assigned
//hours for professor choosen

```

```

String s2 = "UPDATE Professor SET AssignedHours = ? WHERE
LastName = ? ";
//Update the Schedule table. Inserts all data into schedule table
with data
//going into corresponding columns
String s3 = "INSERT INTO Schedule (CourseID, CourseName,
ProfessorID, StartTime, EndTime, Dayf, Room) " +
"VALUES (@CourseID, @CourseName, @ProfessorID, @StartTime, @
EndTime, @Dayf, @Room)";

OleDbCommand updateProfessor = new OleDbCommand(s2, aConnection);
updateProfessor.Parameters.Add(new OleDbParameter("AssignedHours",
, profAsgnedHrs.ToString()));
updateProfessor.Parameters.Add(new OleDbParameter("LastName",
Pname));

OleDbCommand updateCourse = new OleDbCommand(s, aConnection);
updateCourse.Parameters.Add(new OleDbParameter("Assignedhours",
crseAsgnedHrs.ToString()));
updateCourse.Parameters.Add(new OleDbParameter("CourseID",
CourseNum));

//decides room by indication of tab selected
String room;
if (tabControll.SelectedTab == tabPage2)
{
    room = "207";
}
else if (tabControll.SelectedTab == tabPage3)
{
    room = "208";
}
else
{
    room = "203";
}

//Adds parameters to queries
OleDbCommand updateSchedule = new OleDbCommand(s3, aConnection);
updateSchedule.Parameters.Add(new OleDbParameter("@CourseID",
CourseNum));
updateSchedule.Parameters.Add(new OleDbParameter("@CourseName",
CourseName));
updateSchedule.Parameters.Add(new OleDbParameter("@ProfessorID",
Pname));
updateSchedule.Parameters.Add(new OleDbParameter("@StartTime",
Dhour.ToString()));
updateSchedule.Parameters.Add(new OleDbParameter("@EndTime",
Uphour.ToString()));
updateSchedule.Parameters.Add(new OleDbParameter("@Dayf", Datef.
ToString()));
updateSchedule.Parameters.Add(new OleDbParameter("@Room", room.
ToString()));

//Connection is open and queries are ran to update the course,
professor,
//and schedule table
try
{
    aConnection.Open();
    updateCourse.ExecuteNonQuery();
    updateProfessor.ExecuteNonQuery();
    updateSchedule.ExecuteNonQuery();
    aConnection.Close();
}
catch (OleDbException ex)
{

```



```
        MessageBox.Show(ex.Message, "Error(s) Committing Changes");
    }
}
else/// if course has to much hours
{
    MessageBox.Show("The course selected has exceeded it's hours",
        "Exceed Hours");
    //display objects on to currently selected tab
    displaySchedule();
}
}
//reseting values
hx = 0;
hy = 0;
Datef = 0;
Dhour = 0;
Uphour = 0;
}
//*****End of TabPage1 Objects*****

//*****TabPage 2 Objects*****
private void tabPage2_Paint(object sender, PaintEventArgs e)
{
    //redraws graphics when resized
    ResizeRedraw = true;

    //draws graphics for user interface
    Graphics g = e.Graphics;
    Pen p = new Pen(Color.Black, 2);
    int i, j;
    i = j = 60;

    //draws horizontal lines
    while (i < 670)
    {
        Point pt1 = new Point(40, i);
        Point pt2 = new Point((ClientSize.Width - 275), j);
        g.DrawLine(p, pt1, pt2);
        i += 20;
        j += 20;
    }

    Pen pl = new Pen(Color.Blue, 2);
    int k, l;
    k = l = 50;

    //draws vertical lines
    while (k < 560)
    {
        Point pt3 = new Point(k, 0);
        Point pt4 = new Point(l, ClientSize.Height);
        g.DrawLine(pl, pt3, pt4);
        k += 100;
        l += 100;
    }

    Font font = new Font("Arial", 10);

    //Drawing days of the week
    g.DrawString("Monday", font, Brushes.Black, 75, 40);
}
```



```

g.DrawString("Tuesday", font, Brushes.Black, 175, 40);
g.DrawString("Wednesday", font, Brushes.Black, 275, 40);
g.DrawString("Thursday", font, Brushes.Black, 375, 40);
g.DrawString("Friday", font, Brushes.Black, 475, 40);
g.DrawString("Saturday", font, Brushes.Black, 575, 40);
g.DrawString("Sunday", font, Brushes.Black, 675, 40);

int mini = 0;
int houri = 8;
int position = 52;

//Drawing Hours of the Day
while (position < 660)
{
    g.DrawString(houri.ToString() + ":" + mini.ToString() + "0", font,
Brushes.Black, 0, position);
    position += 20;
    mini += 3;
    if (mini == 6)
    {
        houri++;
        mini = 0;
    }
    if (houri == 13)
        houri = 1;
}

//Locates what position mouse is in and fills time & day
int hour_index, date_index;
date_index = (int)((x - 50) / 100) + 1;
hour_index = (int)((y - 60) / 20) + 1;

//Shows position of mouse
//g.DrawString(hour_index.ToString(), font, Brushes.Black, 300,
ClientRectangle.Height - 120);
//g.DrawString(date_index.ToString(), font, Brushes.Black, 400,
ClientRectangle.Height - 120);
//g.DrawString(x.ToString(), font, Brushes.Black, 300, ClientRectangle.Height
- 100);
//g.DrawString(y.ToString(), font, Brushes.Black, 400, ClientRectangle.Height
- 100);
}

private void tabPage2_MouseDown(object sender, MouseEventArgs e)
{
    //calculating start time
    Dhour = (int)((e.Y - 60) / 20) + 1;
    //calculating day of week
    Datef = (int)((e.X - 50) / 100) + 1;
    //calculates points for upper-left corner of highlight
    hx = ((int)((e.X - 50) / 100)) * 100 + 50;
    hy = ((int)((e.Y - 60) / 20)) * 20 + 60;
    //boolean that notifies when mouse is down
    mouse_down = true;
}

private void tabPage2_MouseMove(object sender, MouseEventArgs e)
{
    //retrieves x & y coordinates from mouse event

```

```

        x = e.X;
        y = e.Y;
        //if mouse down highlight form
        if (mouse_down)
        {
            using (Graphics g = tabPage2.CreateGraphics())
            {
                if ((x >= 50) && (y >= 60))
                    g.FillRectangle(Brushes.LightGoldenrodYellow, ((int)((x - 50) /
100)) * 100 + 50, ((int)((y - 60) / 20)) * 20 + 60, 100, 20);
            }
        }

        //*****End of tabPage 2 Objects*****

        //*****TabPage 3 Objects*****
        private void tabPage3_Paint(object sender, PaintEventArgs e)
        {
            //redraws tabpage when resized
            ResizeRedraw = true;

            //Draws Graphics for User Interface
            Graphics g = e.Graphics;
            Pen p = new Pen(Color.Black, 2);
            int i, j;
            i = j = 60;

            //draws horizontal lines
            while (i < 670)
            {
                Point pt1 = new Point(40, i);
                Point pt2 = new Point((ClientSize.Width - 275), j);
                g.DrawLine(p, pt1, pt2);
                i += 20;
                j += 20;
            }

            Pen p1 = new Pen(Color.Blue, 2);
            int k, l;
            k = l = 50;

            //draws vertical lines
            while (k < 560)
            {
                Point pt3 = new Point(k, 0);
                Point pt4 = new Point(l, ClientSize.Height);
                g.DrawLine(p1, pt3, pt4);
                k += 100;
                l += 100;
            }

            Font font = new Font("Arial", 10);

            //Drawing days of the week
            g.DrawString("Monday", font, Brushes.Black, 75, 40);
            g.DrawString("Tuesday", font, Brushes.Black, 175, 40);
            g.DrawString("Wednesday", font, Brushes.Black, 275, 40);
            g.DrawString("Thursday", font, Brushes.Black, 375, 40);
            g.DrawString("Friday", font, Brushes.Black, 475, 40);
            g.DrawString("Saturday", font, Brushes.Black, 575, 40);
            g.DrawString("Sunday", font, Brushes.Black, 675, 40);
        }
    }

```

```

int mini = 0;
int houri = 8;
int position = 52;

//Drawing Hours of the Day
while (position < 660)
{
    g.DrawString(houri.ToString() + ":" + mini.ToString() + "0", font,
Brushes.Black, 0, position);
    position += 20;
    mini += 3;
    if (mini == 6)
    {
        houri++;
        mini = 0;
    }
    if (houri == 13)
        houri = 1;
}

//Locates what position mouse is in and fills time & day
int hour_index, date_index;
date_index = (int)((x - 50) / 100) + 1;
hour_index = (int)((y - 60) / 20) + 1;

//Shows position of mouse
//g.DrawString(hour_index.ToString(), font, Brushes.Black, 500,
ClientRectangle.Height - 120);
//g.DrawString(date_index.ToString(), font, Brushes.Black, 600,
ClientRectangle.Height - 120);
//g.DrawString(x.ToString(), font, Brushes.Black, 500, ClientRectangle.Height
- 100);
//g.DrawString(y.ToString(), font, Brushes.Black, 600, ClientRectangle.Height
- 100);
}

private void tabPage3_MouseDown(object sender, MouseEventArgs e)
{
    //calculating start time
    Dhour = (int)((e.Y - 60) / 20) + 1;
    //calculating day of week
    Datef = (int)((e.X - 50) / 100) + 1;
    //calculates points for upper-left corner of highlight
    hx = ((int)((e.X - 50) / 100)) * 100 + 50;
    hy = ((int)((e.Y - 60) / 20)) * 20 + 60;
    //boolean that notifies when mouse is down
    mouse_down = true;
}

private void tabPage3_MouseMove(object sender, MouseEventArgs e)
{
    //retrieves x & y coordinates from mouse move event
    x = e.X;
    y = e.Y;
    //if mouse down is true will highlight specified region
    if (mouse_down)
    {
        using (Graphics g = tabPage3.CreateGraphics())
        {
            if ((x >= 50) && (y >= 60))

```

```

        g.FillRectangle(Brushes.LightGoldenrodYellow, ((int)((x - 50) /
100)) * 100 + 50, ((int)((y - 60) / 20)) * 20 + 60, 100, 20);
    }
}

private void tabControl1_Click(object sender, EventArgs e)
{
    /*When tab is clicked depending on the corresponding room tabcontrol method
    * will draw all objects associated with that tab(room)
    */

    //if tabpage 2 is selected
    if (tabControl1.SelectedTab == tabPage2)
    {
        //if there are any objects in array
        if (countClass2 > 0)
        {
            int countTab2 = 0;
            //while there are objects display on tabpage
            while (countTab2 <= (countClass2 - 1))
            {
                //if object is null skip
                if (schtrckrArray2[countTab2] == null)
                {
                    countTab2++;
                }
                else
                {
                    using (Graphics g = tabPage2.CreateGraphics())
                    {
                        int recHeight = schtrckrArray2[countTab2].getheight();
                        RectangleF layout = new RectangleF(schtrckrArray2
[countTab2].downx, schtrckrArray2[countTab2].downy, 100, recHeight);
                        g.FillRectangle(Brushes.LightGoldenrodYellow,
schtrckrArray2[countTab2].downx, schtrckrArray2[countTab2].downy, 100, recHeight);
                        Font f = new Font("Arial", 8, FontStyle.Bold);
                        StringFormat format = new StringFormat();
                        format.Alignment = StringAlignment.Center;
                        g.DrawString(schtrckrArray2[countTab2].Prof+"\n" +
schtrckrArray2[countTab2].courseName, f, Brushes.Black, layout, format);
                        countTab2++;
                    }
                }
            }
        }
        else if (tabControl1.SelectedTab == tabPage3)
        {
            if (countClass3 > 0)
            {
                int countTab3 = 0;
                while (countTab3 <= (countClass3 - 1))
                {
                    if (schtrckrArray3[countTab3] == null)
                    {
                        countTab3++;
                    }
                    else
                    {
                        using (Graphics g = tabPage3.CreateGraphics())
                        {
                            int recHeight = schtrckrArray3[countTab3].getheight();
                            RectangleF layout = new RectangleF(schtrckrArray3
[countTab3].downx, schtrckrArray3[countTab3].downy, 100, recHeight);
                            g.FillRectangle(Brushes.LightGoldenrodYellow,

```

```

schtrckrArray3[countTab3].downx, schtrckrArray3[countTab3].downy, 100, recHeight);
    Font f = new Font("Arial", 8, FontStyle.Bold);
    StringFormat format = new StringFormat();
    format.Alignment = StringAlignment.Center;
    g.DrawString(schtrckrArray3[countTab3].Prof+"\n" +
schtrckrArray3[countTab3].courseName, f, Brushes.Black, layout,format);
    countTab3++;
}
}
}
}
else if(tabControl1.SelectedTab == tabPage1)
{
    if (countClass > 0)
    {
        int countTab1 = 0;
        while (countTab1 <= (countClass - 1))
        {
            if (schtrckrArray[countTab1] == null)
            {
                countTab1++;
            }
            else
            {
                using (Graphics g = tabPage1.CreateGraphics())
                {
                    int recHeight = schtrckrArray[countTab1].getheight();
                    RectangleF layout = new RectangleF(schtrckrArray
[countTab1].downx, schtrckrArray[countTab1].downy, 100, recHeight);
                    g.FillRectangle(Brushes.LightGoldenrodYellow,
schtrckrArray[countTab1].downx, schtrckrArray[countTab1].downy, 100, recHeight);
                    Font f = new Font("Arial", 8, FontStyle.Bold);
                    StringFormat format = new StringFormat();
                    format.Alignment = StringAlignment.Center;
                    g.DrawString(schtrckrArray[countTab1].Prof +"\n"+
schtrckrArray[countTab1].courseName, f, Brushes.Black, layout,format);
                    countTab1++;
                }
            }
        }
    }
}

/*The delete button gets information from the double mouse click button, which
 * highlights object that user clicked, and deletes object and redraws the rest
 * of the objects in the corresponding array
 */
private void button2_Click(object sender, EventArgs e)
{
    if(countDelete == -1)
    {
        MessageBox.Show("You did not select a course");
        displaySchedule();
    }
    else
    {
        String Prof, Course;
        double AmtofTime;
        int sTime, eTime, dayf, room, courseNum;

        //depending on which tab is currently selected it will gather data

```

```

//from array and hold info to delete out of schedule table
if (tabControl1.SelectedTab == tabPage2)
{
    Prof = schtrckrArray2[countDelete].Prof;
    Course = schtrckrArray2[countDelete].courseName;
    courseNum = schtrckrArray2[countDelete].courseNum;
    AmtofTime = schtrckrArray2[countDelete].Amtoftime;
    sTime = schtrckrArray2[countDelete].startTime;
    eTime = schtrckrArray2[countDelete].endTime;
    dayf = schtrckrArray2[countDelete].dayDate;
    room = schtrckrArray2[countDelete].roomNum;
}
else if (tabControl1.SelectedTab == tabPage3)
{
    Prof = schtrckrArray3[countDelete].Prof;
    Course = schtrckrArray3[countDelete].courseName;
    courseNum = schtrckrArray3[countDelete].courseNum;
    AmtofTime = schtrckrArray3[countDelete].Amtoftime;
    sTime = schtrckrArray3[countDelete].startTime;
    eTime = schtrckrArray3[countDelete].endTime;
    dayf = schtrckrArray3[countDelete].dayDate;
    room = schtrckrArray3[countDelete].roomNum;
}
else
{
    Prof = schtrckrArray[countDelete].Prof;
    Course = schtrckrArray[countDelete].courseName;
    courseNum = schtrckrArray[countDelete].courseNum;
    AmtofTime = schtrckrArray[countDelete].Amtoftime;
    sTime = schtrckrArray[countDelete].startTime;
    eTime = schtrckrArray[countDelete].endTime;
    dayf = schtrckrArray[countDelete].dayDate;
    room = schtrckrArray[countDelete].roomNum;
}
//handles connection to database
OleDbConnection aConnection = new OleDbConnection("Provider=Microsoft.Jet.
OLEDB.4.0;Data Source=srproj.mdb");
//query to delete selected information out of schedule table
String delComm = "DELETE FROM Schedule WHERE CourseID = ? AND CourseName = ?
AND ProfessorID = ? AND StartTime = ? AND EndTime = ? AND Dayf = ? AND Room = ?";
OleDbCommand updateSchedule = new OleDbCommand(delComm, aConnection);
//parameters in query
updateSchedule.Parameters.Add(new OleDbParameter("CourseID", courseNum));
updateSchedule.Parameters.Add(new OleDbParameter("CourseName", Course));
updateSchedule.Parameters.Add(new OleDbParameter("ProfessorID", Prof));
updateSchedule.Parameters.Add(new OleDbParameter("StartTime", sTime));
updateSchedule.Parameters.Add(new OleDbParameter("EndTime", eTime));
updateSchedule.Parameters.Add(new OleDbParameter("Dayf", dayf));
updateSchedule.Parameters.Add(new OleDbParameter("Room", room));
//deletes selected information out of schedule table in database
try
{
    aConnection.Open();
    updateSchedule.ExecuteNonQuery();
}
catch (OleDbException ex)
{
    MessageBox.Show(ex.Message, "Error(s) Committing Schedule Changes");
}
finally
{
    aConnection.Close();
}

double currCreHr = 0;

```

```

double currPrfHr = 0;
//retrieving hours assigned house from course and professor table
OleDbCommand getCouse = new OleDbCommand("select * from Course Where
CourseID = ?", aConnection);
getCourse.Parameters.Add(new OleDbParameter("CourseID",courseNum));
OleDbCommand getProf = new OleDbCommand("Select * from Professor Where
LastName = ?", aConnection);
getProf.Parameters.Add(new OleDbParameter("LastName",Prof));

try
{
    aConnection.Open();
    OleDbDataReader readCourse = getCourse.ExecuteReader();
    OleDbDataReader readProf = getProf.ExecuteReader();
    while (readCourse.Read())
    {
        currCreHr = readCourse.GetDouble(4);
    }
    readCourse.Close();
    while (readProf.Read())
    {
        currPrfHr = readProf.GetDouble(4);
    }
    readProf.Close();
}
catch (OleDbException ex)
{
    Console.WriteLine("Error: {0}", ex.Errors[0].Message);
}
finally
{
    aConnection.Close();
}

//subtracting hours from course and professor
currCreHr = currCreHr - AmtofTime;
currPrfHr -= AmtofTime;

//Updating assigned hours for course and professor
OleDbCommand updateCourse = new OleDbCommand("UPDATE Course SET Assignedhours
= ? " +
    " Where CourseID = ?", aConnection);
updateCourse.Parameters.Add(new OleDbParameter("Assignedhour", currCreHr));
updateCourse.Parameters.Add(new OleDbParameter("CourseID", courseNum));

OleDbCommand updateProf = new OleDbCommand("UPDATE Professor SET
AssignedHours = ?" +
    " Where LastName = ?", aConnection);
updateProf.Parameters.Add(new OleDbParameter("AssignedHours", currPrfHr));
updateProf.Parameters.Add(new OleDbParameter("LastName", Prof));

try
{
    aConnection.Open();
    updateProf.ExecuteNonQuery();
    updateCourse.ExecuteNonQuery();
}
catch (OleDbException ex)
{
    Console.WriteLine("Error: {0}", ex.Errors[0].Message);
}
finally
{
    aConnection.Close();
}

```



```

//Redraws objects back on to tabPage depending on which
//tabpage is selected
if (tabControl1.SelectedTab == tabPage2)
{
    schtrckrArray2[countDelete] = null;
    tabPage2.Refresh();
    int countTab2 = 0;
    while (countTab2 < 20)
    {
        if (schtrckrArray2[countTab2] == null)
            countTab2++;
        else
        {
            using (Graphics g = tabPage2.CreateGraphics())
            {
                int recHeight = schtrckrArray2[countTab2].getheight();
                RectangleF layout = new RectangleF(schtrckrArray2[countTab2].
downx, schtrckrArray2[countTab2].downy, 100, recHeight);
                g.FillRectangle(Brushes.LightGoldenrodYellow, schtrckrArray2
[countTab2].downx, schtrckrArray2[countTab2].downy, 100, recHeight);
                Font f = new Font("Arial", 8, FontStyle.Bold);
                StringFormat format = new StringFormat();
                format.Alignment = StringAlignment.Center;
                g.DrawString(schtrckrArray2[countTab2].Prof+"\n" +
schtrckrArray2[countTab2].courseName, f, Brushes.Black, layout,format);
            }
            countTab2++;
        }
    }
}
else if (tabControl1.SelectedTab == tabPage3)
{
    schtrckrArray3[countDelete] = null;
    tabPage3.Refresh();
    int countTab3 = 0;
    while (countTab3 < 20)
    {
        if (schtrckrArray3[countTab3] == null)
            countTab3++;
        else
        {
            using (Graphics g = tabPage3.CreateGraphics())
            {
                int recHeight = schtrckrArray3[countTab3].getheight();
                RectangleF layout = new RectangleF(schtrckrArray3[countTab3].
downx, schtrckrArray3[countTab3].downy, 100, recHeight);
                g.FillRectangle(Brushes.LightGoldenrodYellow, schtrckrArray3
[countTab3].downx, schtrckrArray3[countTab3].downy, 100, recHeight);
                Font f = new Font("Arial", 8, FontStyle.Bold);
                StringFormat format = new StringFormat();
                format.Alignment = StringAlignment.Center;
                g.DrawString(schtrckrArray3[countTab3].Prof+"\n" +
schtrckrArray3[countTab3].courseName, f, Brushes.Black, layout,format);
            }
            countTab3++;
        }
    }
}
else
{
    schtrckrArray[countDelete] = null;
    tabPage1.Refresh();
    int countTab = 0;
    while (countTab < 20)

```



```

    {
        if (schtrckrArray[countTab] == null)
        {
            countTab++;
        }
        else
        {
            using (Graphics g = tabPage1.CreateGraphics())
            {
                int recHeight = schtrckrArray[countTab].getheight();
                RectangleF layout = new RectangleF(schtrckrArray[countTab].
downx, schtrckrArray[countTab].downy, 100, recHeight);
                g.FillRectangle(Brushes.LightGoldenrodYellow, schtrckrArray
[countTab].downx, schtrckrArray[countTab].downy, 100, recHeight);
                Font f = new Font("Arial", 8, FontStyle.Bold);
                StringFormat format = new StringFormat();
                format.Alignment = StringAlignment.Center;
                g.DrawString(schtrckrArray[countTab].Prof+"\n" +
schtrckrArray[countTab].courseName, f, Brushes.Black, layout, format);
            }
            countTab++;
        }
    }
}

/*Activates when user clicks on an object that the user wants to delete
 * Method searches and makes sure that the item is selected and then
 * highlights the object in read
 * */

private void tabPage1_MouseDoubleClick(object sender, MouseEventArgs e)
{
    //retrieving x & y for mouse double click event
    int count = 0;
    int xdc = e.X;
    int ydc = e.Y;

    //if the are objects in the array it searches for the object that the
    //user clicked on
    while (count < 20)
    {
        if (schtrckrArray[count] == null)
            count++;
        else
        {
            if (xdc > schtrckrArray[count].downx && xdc < schtrckrArray[count].
upxc &&
                ydc > schtrckrArray[count].downy && ydc < schtrckrArray[count].
upyc)
            {
                countDelete = count;
                using (Graphics g = tabPage1.CreateGraphics())
                {
                    int recHeight = schtrckrArray[countDelete].getheight();
                    RectangleF layout = new RectangleF(schtrckrArray[countDelete]
.downx, schtrckrArray[countDelete].downy, 100, recHeight);
                    g.FillRectangle(Brushes.Firebrick, schtrckrArray[countDelete]
.downx, schtrckrArray[countDelete].downy, 100, recHeight);
                    Font f = new Font("Arial", 8, FontStyle.Bold);
                    StringFormat format = new StringFormat();
                    format.Alignment = StringAlignment.Center;
                    g.DrawString(schtrckrArray[countDelete].Prof+"\n" +

```

```

schtrckrArray[countDelete].courseName, f, Brushes.Black, layout,format);

        }
        break;
    }
    else
    {
        count++;
    }
}

}

/*Activates when user clicks on an object that the user wants to delete
 * Method searches and makes sure that the item is selected and then
 * highlights the object in read
 * */

private void tabPage2_MouseDoubleClick(object sender, MouseEventArgs e)
{
    //retrieving x & y for mouse double click event
    int count = 0;
    int xdc = e.X;
    int ydc = e.Y;

    //if the are objects in the array it searches for the object that the
    //user clicked on
    while (count < 20)
    {
        if (schtrckrArray2[count] == null)
            count++;
        else
        {
            if (xdc > schtrckrArray2[count].downx && xdc < schtrckrArray2[count].↵
upxc &&
            ydc > schtrckrArray2[count].downy && ydc < schtrckrArray2[count].↵
upyc)
            {
                countDelete = count;
                using (Graphics g = tabPage2.CreateGraphics())
                {
                    int recHeight = schtrckrArray2[countDelete].getheight();
                    RectangleF layout = new RectangleF(schtrckrArray2
[countDelete].downx, schtrckrArray2[countDelete].downy, 100, recHeight);
                    g.FillRectangle(Brushes.Firebrick, schtrckrArray2
[countDelete].downx, schtrckrArray2[countDelete].downy, 100, recHeight);
                    Font f = new Font("Arial", 8, FontStyle.Bold);
                    StringFormat format = new StringFormat();
                    format.Alignment = StringAlignment.Center;
                    g.DrawString(schtrckrArray2[countDelete].Prof+"\n" +
schtrckrArray2[countDelete].courseName, f, Brushes.Black, layout,format);

                }
                break;
            }
            else
            {
                count++;
            }
        }
    }
}

```

```

    }
}

private void tabPage2_MouseUp(object sender, MouseEventArgs e)
{
    //calculates end time
    Uphour = (int)((e.Y - 60) / 20) + 1;
    //calculates right hand corner of object
    bttmx = ((int)((e.X - 50) / 100)) * 100 + 50 + 100;
    bttmy = ((int)((e.Y - 60) / 20)) * 20 + 60 + 20;
    //boolean that is false when mouse up event is fired
    mouse_down = false;
}

private void tabPage3_MouseUp(object sender, MouseEventArgs e)
{
    //calculates end time
    Uphour = (int)((e.Y - 60) / 20) + 1;
    //calculates right hand corner of object
    bttmx = ((int)((e.X - 50) / 100)) * 100 + 50 + 100;
    bttmy = ((int)((e.Y - 60) / 20)) * 20 + 60 + 20;
    //boolean that is false when mouse up event is fired
    mouse_down = false;
}

/*Activates when user clicks on an object that the user wants to delete
* Method searches and makes sure that the item is selected and then
* highlights the object in read
* */
private void tabPage3_MouseDoubleClick(object sender, MouseEventArgs e)
{
    //retrieving x & y for mouse double click event
    int count = 0;
    int xdc = e.X;
    int ydc = e.Y;

    //if the are objects in the array it searches for the object that the
    //user clicked on
    while (count < 20)
    {
        if (schtrckrArray3[count] == null)
            count++;
        else
        {
            if (xdc > schtrckrArray3[count].downx && xdc < schtrckrArray3[count].
upxc &&
ydc > schtrckrArray3[count].downy && ydc < schtrckrArray3[count].
upyc)
            {
                countDelete = count;
                using (Graphics g = tabPage3.CreateGraphics())
                {
                    int recHeight = schtrckrArray3[countDelete].getheight();
                    RectangleF layout = new RectangleF(schtrckrArray3
[countDelete].downx, schtrckrArray3[countDelete].downy, 100, recHeight);
                    g.FillRectangle(Brushes.Firebrick, schtrckrArray3
[countDelete].downx, schtrckrArray3[countDelete].downy, 100, recHeight);
                    Font f = new Font("Arial", 8, FontStyle.Bold);
                    StringFormat format = new StringFormat();
                    format.Alignment = StringAlignment.Center;
                    g.DrawString(schtrckrArray3[countDelete].Prof+"\n" +
schtrckrArray3[countDelete].courseName, f, Brushes.Black, layout, format);
                }
            }
        }
    }
}

```

```

        }
        break;

    }
    else
    {
        count++;
    }
}

}

/*When these button is clicked it deletes all information from all arrays
 * and also deletes everything out of schedule table and sets values
 * related to arrays to zero
 * */

private void button3_Click(object sender, EventArgs e)
{
    //ask user if they want to delete all information
    DialogResult result = MessageBox.Show("Are you sure you want to delete ALL
scheduled courses?", "Delete All", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
    //if yes proceed to delete everything
    if (result == DialogResult.Yes)
    {
        int count = 0;
        //deletes all objects out of arrays
        while (count < 20)
        {
            schtrckrArray[count] = null;
            schtrckrArray2[count] = null;
            schtrckrArray3[count] = null;
            count++;
        }
        countClass = 0;
        countClass2 = 0;
        countClass3 = 0;

        //depending on the tabpage selected, redraws graphics
        if (tabControll1.SelectedTab == tabPage2)
            tabPage2.Refresh();
        else if (tabControll1.SelectedTab == tabPage3)
            tabPage3.Refresh();
        else
            tabPage1.Refresh();

        double zherod = 0;

        //Query that deletes everything out of the schedule table and also sets
assigned hours
        // for professors and course to zero
        OleDbConnection aConnection = new OleDbConnection("Provider=Microsoft.Jet.
OLEDB.4.0;Data Source=srproj.mdb");
        OleDbCommand clearSchedule = new OleDbCommand("Delete From Schedule",
aConnection);
        OleDbCommand clearCourse = new OleDbCommand("UPDATE Course SET
Assignedhours = ? WHERE CourseID > ?", aConnection);
        clearCourse.Parameters.Add(new OleDbParameter("Assignedhours", zherod));
        clearCourse.Parameters.Add(new OleDbParameter("CourseID", "200"));
        OleDbCommand clearProf = new OleDbCommand("UPDATE Professor SET
AssignedHours = ? WHERE ProfessorID >= ?", aConnection);
        clearProf.Parameters.Add(new OleDbParameter("AssignedHours", zherod));
        clearProf.Parameters.Add(new OleDbParameter("ProfessorID", "1"));

        try
        {
            aConnection.Open();

```

```

        clearSchedule.ExecuteNonQuery();
        clearProf.ExecuteNonQuery();
        clearCourse.ExecuteNonQuery();
    }
    catch (OleDbException ex)
    {
        Console.WriteLine("Error: {0}", ex.Errors[0].Message);
    }
    finally
    {
        aConnection.Close();
    }
}
else//if user doesnt want to delete everything
    MessageBox.Show("Courses were not erased");
}
//Undo button refreshes tabpage according to the tabpage that is currently
selected
private void button4_Click(object sender, EventArgs e)
{
    displaySchedule();
}

//DisplaySchedule method draws the all objects onto the current tabpage that
//is selected
public void displaySchedule()
{
    //found out which tabpage is selected
    int room;
    if (tabControl1.SelectedTab == tabPage2)
        room = 207;
    else if (tabControl1.SelectedTab == tabPage3)
        room = 208;
    else
        room = 203;

    //depending on the tabpage selected will display all objects
    //associated with that tabpage
    switch (room)
    {
        case 203:
            tabPage1.Refresh();
            if (countClass > 0)
            {
                int countTab1 = 0;
                while (countTab1 <= (countClass - 1))
                {
                    if (schtrckrArray[countTab1] == null)
                    {
                        countTab1++;
                    }
                    else
                    {
                        using (Graphics g = tabPage1.CreateGraphics())
                        {
                            int recHeight = schtrckrArray[countTab1].getheight();
                            RectangleF layout = new RectangleF(schtrckrArray
[countTab1].downx, schtrckrArray[countTab1].downy, 100, recHeight);
                            g.FillRectangle(Brushes.LightGoldenrodYellow,
schtrckrArray[countTab1].downx, schtrckrArray[countTab1].downy, 100, recHeight);
                            Font f = new Font("Arial", 8, FontStyle.Bold);
                            StringFormat format = new StringFormat();
                            format.Alignment = StringAlignment.Center;
                            g.DrawString(schtrckrArray[countTab1].Prof + "\n" +
schtrckrArray[countTab1].courseName, f, Brushes.Black, layout, format);
                            countTab1++;
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
    }
    }
    break;
case 207:
    tabPage2.Refresh();
    if (countClass2 > 0)
    {
        int countTab2 = 0;
        while (countTab2 <= (countClass2 - 1))
        {
            if (schtrckrArray2[countTab2] == null)
            {
                countTab2++;
            }
            else
            {
                using (Graphics g = tabPage2.CreateGraphics())
                {
                    int recHeight = schtrckrArray2[countTab2].getheight() ✓
;
                    RectangleF layout = new RectangleF(schtrckrArray2 ✓
[countTab2].downx, schtrckrArray2[countTab2].downy, 100, recHeight);
                    g.FillRectangle(Brushes.LightGoldenrodYellow, ✓
schtrckrArray2[countTab2].downx, schtrckrArray2[countTab2].downy, 100, recHeight);
                    Font f = new Font("Arial", 8, FontStyle.Bold);
                    StringFormat format = new StringFormat();
                    format.Alignment = StringAlignment.Center;
                    g.DrawString(schtrckrArray2[countTab2].Prof+"\n" + ✓
schtrckrArray2[countTab2].courseName, f, Brushes.Black, layout, format);
                    countTab2++;
                }
            }
        }
    }
    break;
case 208:
    tabPage3.Refresh();
    if (countClass3 > 0)
    {
        int countTab3 = 0;
        while (countTab3 <= (countClass3 - 1))
        {
            if (schtrckrArray3[countTab3] == null)
            {
                countTab3++;
            }
            else
            {
                using (Graphics g = tabPage3.CreateGraphics())
                {
                    int recHeight = schtrckrArray3[countTab3].getheight() ✓
;
                    RectangleF layout = new RectangleF(schtrckrArray3 ✓
[countTab3].downx, schtrckrArray3[countTab3].downy, 100, recHeight);
                    g.FillRectangle(Brushes.LightGoldenrodYellow, ✓
schtrckrArray3[countTab3].downx, schtrckrArray3[countTab3].downy, 100, recHeight);
                    Font f = new Font("Arial", 8, FontStyle.Bold);
                    StringFormat format = new StringFormat();
                    format.Alignment = StringAlignment.Center;
                    g.DrawString(schtrckrArray3[countTab3].Prof+"\n" + ✓
schtrckrArray3[countTab3].courseName, f, Brushes.Black, layout, format);
                    countTab3++;
                }
            }
        }
    }

```

```

    }
    }
    break;
default:
    break;
}

//checkSchedule method checks to make sure that a professor does not have
//a conflict in his schedule
public bool checkSchedule(scheduletrckr[] sArray, String Prof, int Day, int
startTime, int countC, String CourseNam)
{
    //if there are objects in the array iterate through the array
    if (countC != 0)
    {
        int c = 0;
        while (c < 20)
        {
            if (sArray[c] == null)
            {
                c++;
            }
            else
            {
                //checks to see if course conflicts with another course
                if ((sArray[c].Prof == Prof && sArray[c].dayDate == Day)
                    && (sArray[c].endTime >= startTime))
                {
                    //if there is a conflict user is notified and method
                    //returns true
                    MessageBox.Show("There is a conflict between "
                        + sArray[c].courseName + " and " + CourseNam, "Conflict"
, MessageBoxButtons.OK,
                    MessageBoxIcon.Exclamation);
                    return true;
                }

                c++;
            }
        }
        return false;
    }
    else
        return false;
}

//displays schedule if user resizes form
private void Form1_ResizeEnd(object sender, EventArgs e)
{
    displaySchedule();
}

//displays schedule when form is first opened
private void Form1_Shown(object sender, EventArgs e)
{
    displaySchedule();
}

//displays schedule when form is activated
private void Form1_Activated(object sender, EventArgs e)
{
    displaySchedule();
}

```



```
    }  
    private void Form1_SizeChanged(object sender, EventArgs e)  
    {  
        sized_changed = true;  
    }
```

```
}
```

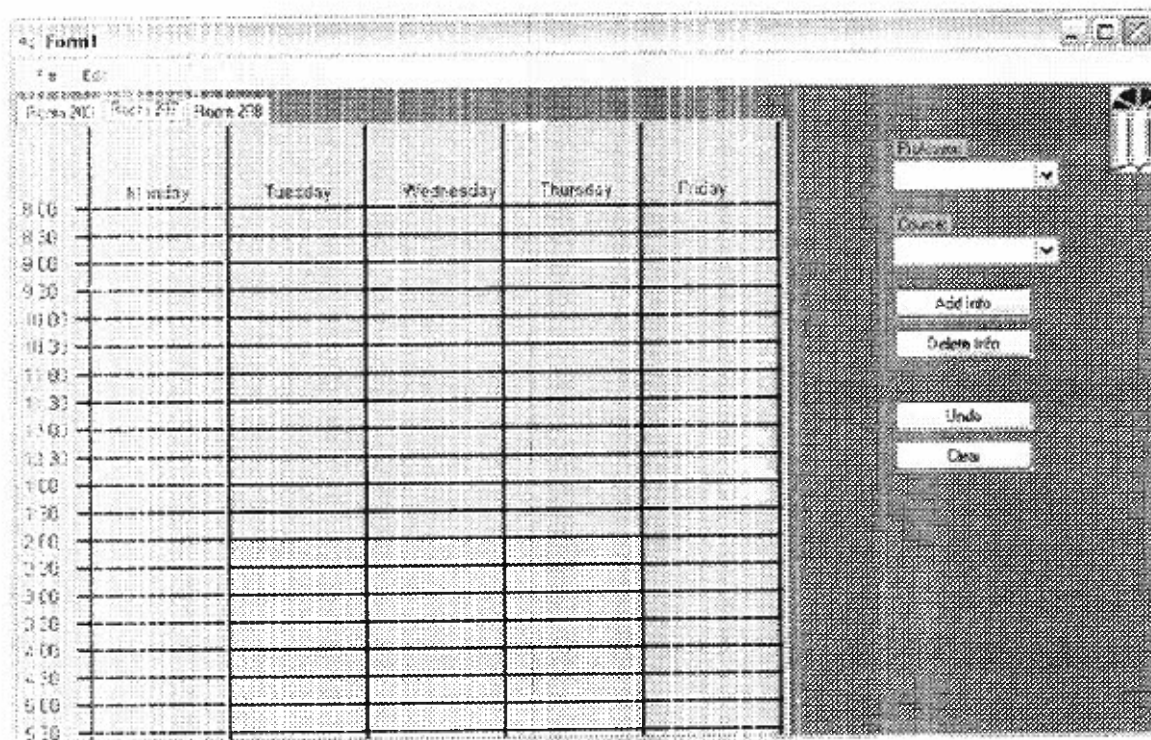
```
}
```

New Users Guide for Scheduler Program

I. Add Information

a. Select a room

- i. Click on the corresponding tab to select a room



b. Highlight the time slot that

- i. Highlight the time slot by clicking and holding down the left mouse button at the desired start time
- ii. Drag the mouse down until you have reached your desired end time

File Edit View Help

Room: 00 Room: 00 Room: 00 Room: 00 Room: 00

Room: 00 Room: 00 Room: 00 Room: 00 Room: 00

	Monday	Tuesday	Wednesday	Thursday	Friday
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					
11:30					
12:00					
12:30					
1:00					
1:30					
2:00					
2:30					
3:00					
3:30					
4:00					
4:30					
5:00					
5:30					
6:00					
6:30					
7:00					
7:30					
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					

Professor: [v]
 Course: [v]
 Add Info
 Delete Info
 Login
 Close

CHARLESTON SOUTHERN UNIVERSITY

c. Select Professor by clicking on the Professor combo box

File Edit View Help

Room: 00 Room: 00 Room: 00 Room: 00 Room: 00

Room: 00 Room: 00 Room: 00 Room: 00 Room: 00

	Monday	Tuesday	Wednesday	Thursday	Friday
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					
11:30					
12:00					
12:30					
1:00					
1:30					
2:00					
2:30					
3:00					
3:30					
4:00					
4:30					
5:00					
5:30					
6:00					
6:30					
7:00					
7:30					
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					

Professor: [v]
 Course: [v]
 Add Info
 Delete Info
 Login
 Close

CHARLESTON SOUTHERN UNIVERSITY

d. Select Course by clicking on the Course combo box

The screenshot displays a web application for Charleston Southern University. On the left, there is a grid for selecting courses, with columns for 'Mon', 'Tue', 'Wed', 'Thurs', and 'Fri'. The rows represent time slots from 8:00 to 11:30. On the right, there is a 'Course' dropdown menu that is currently open, showing a list of course options. The university's logo is visible in the top right corner. The interface is designed for users to select a course and then click an 'Add' button to register for it.

e. Click on the add button once you have chosen the desired Professor and Course

Form 1
12

Page:

Ln:

Cash:

212

Auto Pay

Paid by:

Card

Clear

	Sunday	Monday	Tuesday	Wednesday	Thursday	Friday
11:00						
11:30						
12:00						
12:30						
1:00						
1:30						
2:00						
2:30						
3:00						
3:30						
4:00						
4:30						
5:00						
5:30						
6:00						
6:30						
7:00						
7:30						
8:00						
8:30						
9:00						
9:30						
10:00						
10:30						
11:00						
11:30						
12:00						
12:30						
1:00						
1:30						
2:00						
2:30						
3:00						
3:30						
4:00						
4:30						
5:00						
5:30						
6:00						
6:30						
7:00						
7:30						
8:00						
8:30						
9:00						
9:30						
10:00						
10:30						
11:00						
11:30						
12:00						
12:30						
1:00						
1:30						
2:00						
2:30						
3:00						
3:30						
4:00						
4:30						
5:00						
5:30						
6:00						
6:30						
7:00						
7:30						
8:00						
8:30						
9:00						
9:30						
10:00						
10:30						
11:00						
11:30						
12:00						
12:30						
1:00						
1:30						
2:00						
2:30						
3:00						
3:30						
4:00						
4:30						
5:00						
5:30						
6:00						
6:30						
7:00						
7:30						
8:00						
8:30						
9:00						
9:30						
10:00						
10:30						
11:00						
11:30						
12:00						
12:30						
1:00						

II. Delete Information

2. **Select Time Slot**

- i. Select Time Slot by double clicking the desired time slot
- ii. Once double clicked the Time Slot chosen will be highlighted in red

Form

File Edit

Rooms: 200 Room 207 Room 208

	Monday	Tuesday	Wednesday	Thursday	Friday
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					
11:30					
12:00					
12:30					
1:00					
1:30					
2:00					
2:30					
3:00					
3:30					
4:00					
4:30					
5:00					
5:30					
6:00					
6:30					
7:00					
7:30					
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					
11:30					

Professor:

Course:

Add Info

Delete Info

Undo

Clear

CHARLESTON SOUTHERN UNIVERSITY

- iii. Click the delete information button and the information will be erased

Form1

File Edit

Rooms: 200 Room 207 Room 208

	Monday	Tuesday	Wednesday	Thursday	Friday
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					
11:30					
12:00					
12:30					
1:00					
1:30					
2:00					
2:30					
3:00					
3:30					
4:00					
4:30					
5:00					
5:30					

Professor:

Course:

Add Info

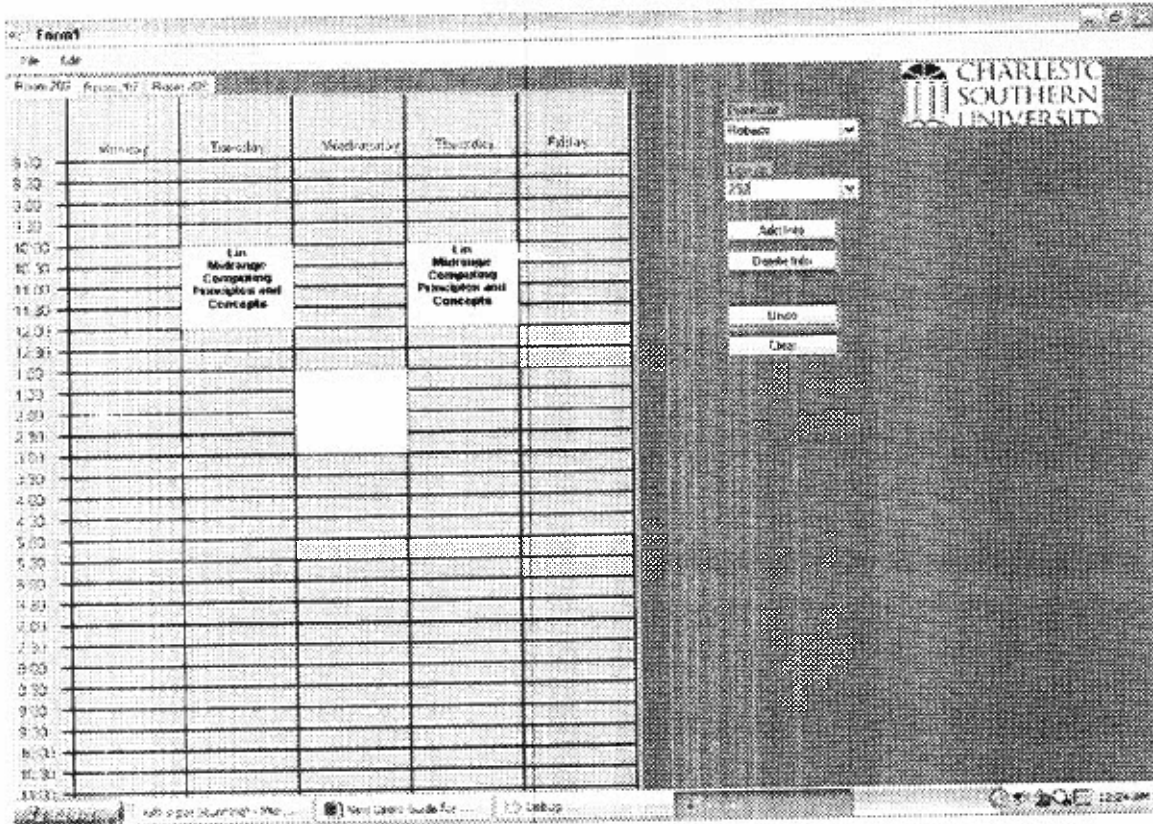
Delete Info

Undo

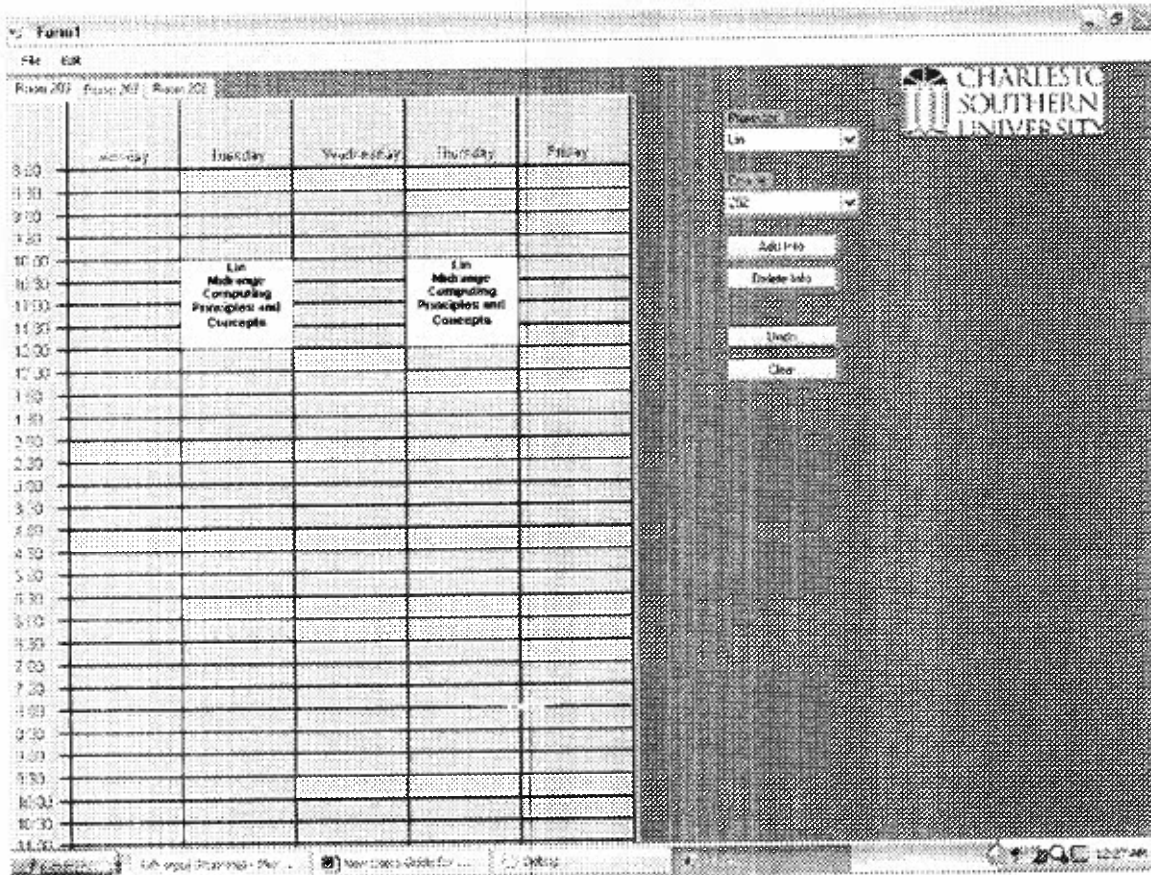
Clear

III. Undo Button

- a. The undo button is useful in two situations:
 - i. To clear a highlighted time slot before the add information button is clicked



After clear button is clicked



ii. To clear a highlighted time slot that is selected to be deleted

Before Undo is Clicked

Form1

File Edit

Room: 200 Room: 201 Room: 202

	Monday	Tuesday	Wednesday	Thursday	Friday
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					
11:30					
12:00					
12:30					
1:00					
1:30					
2:00					
2:30					
3:00					
3:30					
4:00					
4:30					
5:00					
5:30					
6:00					
6:30					
7:00					
7:30					
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					

Lin
Mid-range
Computing
Principles and
Concepts

CHARLESTON
SOUTHERN
UNIVERSITY

Room: 200
Lin
Course: 200
Add Info
Delete Info
Undo
Clear

12:00 AM

After Undo is Clicked

Form1

File Edit

Room: 200 Room: 201 Room: 202

	Monday	Tuesday	Wednesday	Thursday	Friday
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					
11:30					
12:00					
12:30					
1:00					
1:30					
2:00					
2:30					
3:00					
3:30					
4:00					
4:30					
5:00					
5:30					
6:00					
6:30					
7:00					
7:30					
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					

Lin
Mid-range
Computing
Principles and
Concepts

CHARLESTON
SOUTHERN
UNIVERSITY

Room: 200
Lin
Course: 200
Add Info
Delete Info
Undo
Clear

12:00 AM

-

Form1

File Edit

Room 203 Room 207 Room 208

	Monday	Tuesday	Wednesday	Thursday	Friday
8:00					
8:30					
9:00					
9:30					
10:00					
10:30					
11:00					
11:30					
12:00					
12:30					
1:00					
1:30					
2:00					
2:30					
3:00					
3:30					
4:00					
4:30					
5:00					
5:30					

Professor

Course

Add Info

Delete Info

Undo

Clear

I formed a class to handle the graphics objects for the user interface. I felt that it was necessary to keep track of all the graphics objects that were added to the form and to me the best way to do it was to add a class that held everything that was need for the graphics objects. Whether it be information to draw the graphics object onto the page or for me to use it to help access information in the database.

```
public class scheduletrckr
```

```
{
```

The attributes used in the class were Prof, which held the professors last name.

```
public String Prof;
```

```
//Course Name and Course Number and Length of Course
```

CourseName held the name for the course

```
public String courseName;
```

CourseNum held the course number for the course

```
public int courseNum;
```

AmtofTime was used to calculate the length of the course

```
public double Amtoftime;
```

downx and downy came from the mouse down event. These help indicate the start time of the course and was also used to draw the graphics

object. They were the upper-left corner of the rectangle in the graphics object

```
//X & Y Coordinates for upper left corner of highlighted region  
public int downx;  
public int downy;  
  
// X & Y Coordinate for bottom right corner of highlighted  
region
```

upxc and upyc came from the mouse up event. These help indicate the end time of the course. Upyc was used to calculate the height of the rectangle. They were also the bottom right coordinates of the graphics object

```
public int upxc;  
public int upyc;  
  
//start time & end time of course
```

Start time was calculated using downy

```
public int startTime;
```

end time was calculated using upyc

```
public int endTime;  
  
//Day of week for course and room number
```

dayDate was a variable that held the day of the week. It was calculated by using the downx variable the dayDate is represented by numbers i.e. 1 = Monday 2 = Tuesday... and so on

```
public int dayDate;
```

Room indicated which room number

```
public int roomNum;
```

The following constructor took in data from the interface that the user generated. The constructor took in the professors last name, the course number, the course name, the length of the course, upper left corner x & y coordinates for graphics object, and the bottom right corner x & y coordinates

```
    //Constructor that sets attributes in object
    //Used when adding a new object to the array
    public scheduletrckr(String ProfA, String Course, int CourseN,
double AmtTime, int dwx, int dwy, int upx, int upy)
    {
Set the attributes for the new object
```

```
        Prof = ProfA;
        courseName = Course;
        courseNum = CourseN;
        Amtoftime = AmtTime;
        downx = dwx;
        downy = dwy;
        upxc = upx;
        upyc = upy;

        //Calculates start time, end time, day of week
        setAttributes2();
```

```
    }
```

These is another constructor that was used when the form first loaded.
The objects were created by getting information from the schedule table
in the database

```
//Constructor that sets attributes in object
//used when bringing in data from database
public scheduletrckr(String ProfA, String Course,int CourseN,
int sTime, int eTime, int dDate, int room)
{
    Prof = ProfA;
    courseName = Course;
    courseNum = CourseN;
    startTime = sTime;
    endTime = eTime;
    dayDate = dDate;
    roomNum = room;
    setAttributes();
}
```

calculates height for rectangle graphic object. Used when drawing
graphics object

```
// calculates and sets height for object
public int getheight()
{
    since height is controlled by the y coordinate. I took the bottom
    right y and subtracted by the upper y, which in result gave me my
    height

    int heightfrrec = upyc - downy;
```



```
return heightfrrec;
```

setAttributes function runs in the constructor that gets data from the database on form load. It calculates the downx, downy, upyc, and upxc

```
// calculates and sets X & Y coordinates for upper-left and  
bottom right
```

```
// of object and calculates length of course
```

```
public void setAttributes()
```

```
{
```

I wanted to get to get the downx so I have coordinates to draw graphics object. To do that I reversed the equation that gives the day of the week in a numerical representation. I will explain why I used the numbers later

```
int natx = ((dayDate - 1) * 100) + 50;
```

Once I converted the dayDate back to its natural coordinate I then proceed to calculate what downx would be

```
downx = (((natx - 50) / 100)) * 100 + 50;
```

I had to convert the start time as well to get the original y coordinate from the mouse down event

```
int naty = ((startTime - 1) * 20) + 60;
```

```
downy = (((naty - 60) / 20)) * 20 + 60;
```

```
int naty2 = ((endTime - 1) * 20) + 60;
```

I also calculate the coordinates for the mouseup button, which are the coordinates from the bottom right corner of the graphics object

```
upyc = (((naty2 - 60) / 20)) * 20 + 60 + 20;
```

```
int natx2 = ((dayDate - 1) / 100) + 50;
```

```
upxc = downx+100;
```

```
        Amtoftime = (((double)endTime - (double)startTime) / 2 +  
.5);
```

The function setAttributes2 was used to set the start time, end time and the day of the week. I made this function so the add button was able to check if a professor had a conflicting course.

```
//calculates and sets start time, end time, and day of the week
```

```
public void setAttributes2()  
{  
    int natx = ((downx - 50)/100) * 100 + 50;  
    dayDate = ((natx - 50) / 100) + 1;  
    int naty = ((downy - 60) / 20) * 20 + 60;  
    startTime = ((naty - 60) / 20) + 1;  
    int naty2 = (((upyc - 20) - 60) / 20) * 20 + 60;  
    endTime = ((naty2 - 60) / 20) + 1;  
}
```

Global Variables

These variables are global because I didn't know how to incorporate them into the program any other way. Since this was my first time programming in C# I decided to go these route.

Hx and hy were used to capture the x & y coordinates from the mouse down event

```
int x, y, hx, hy;  
  
    int Datef, Dhour, Uphour;  
  
    String Lname, Fname;  
  
    double hour;
```

mouse down boolean variable was used to help notify the system when I wanted a region to be highlighted

```
    bool mouse_down = false;  
  
    bool sized_changed = false;
```

The scheduletrckr arrays were used to hold objects of type scheduletrckr. I thought this was a great way of keeping the graphics separated and grouped by which room the objects belonged too.

```
    //data structure used to keep track of schedule for each  
room(tab)
```

Associated with room 203

```
    scheduletrckr[] schtrckrArray = new scheduletrckr[20];
```

Associated with room 207

```
    scheduletrckr[] schtrckrArray2 = new scheduletrckr[20];
```

Associated with room 208

```
    scheduletrckr[] schtrckrArray3 = new scheduletrckr[20];
```

```
    //Keep track of how many objects are in the array
```

The countclass variables were used to keep track of how many objects were in each array

Associated with schtrckArray(room 203)

```
int countClass = 0;
```

Associated with schtrckArray2(room 207)

```
int countClass2 = 0;
```

Associated with schtrckArray3 (room 208)

```
int countClass3 = 0;
```

```
//x & y for mouse up event
```

Used to capture mouse up event, which were the bottom right corner coordinates for the graphics object.

```
int bttmx, bttmy;
```

```
//keeps track of the object that is to be deleted
```

countDelete was used to keep track of the object that is to get deleted from an array.

```
int countDelete;
```

Form Load Method

In the form load method I mainly used it for bringing in the data from the schedule table and putting it into the corresponding array .

```
private void Form1_Load(object sender, EventArgs e)
```

```
{
```

This set the properties of the tabcontrol which contained the tabpages

```
//Setting attributes for tabcontrol
```

```
tabControl1.Anchor = AnchorStyles.Top | AnchorStyles.Left |
```

```
AnchorStyles.Bottom;
```

```
tabControl1.Size = new Size(560, ClientSize.Height);
```

```
//sets picture in right hand corner of form
pictureBox1.Image = Image.FromFile("CSUlogo.jpg");
```

```
//*****Connection to
database*****\\
//*****
****\\
```

//Handles the connection to database and also queries the
table needing to be accessed

The following line creates the line of communication to the database

```
OleDbConnection aConnection = new
OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;Data
Source=srproj.mdb");
```

The following line is the query that was run to access data from the
database. It is selecting the Professor table

```
OleDbCommand aCommand = new OleDbCommand("select * from
Professor", aConnection);
```

The following line is the query that was run to access data from the
course table in the database

```
OleDbCommand aCommand1 = new OleDbCommand("select * from
Course", aConnection);
```

These connects C# to the access database

```
//Connecting to database
try
{
```

open the connection to the database

```
aConnection.Open();
```

The following lines created a data reader to read from the table that the queries are accessing

```
//create the datareader object to connect to table  
OleDbDataReader aReader = aCommand.ExecuteReader();  
OleDbDataReader aReader1 = aCommand1.ExecuteReader();
```

The following line goes through the table reading data that the query has asked for while there is data that meets the queries demands

```
//Iterate through the database Professor table  
while (aReader.Read())  
{  
    areadr.getString(1) retrieves data from the second column in the  
    professor table, which is the professors last name  
  
    //adds professor names to combo box  
    Lname = aReader.GetString(1);  
    comboBox1.Items.Add(Lname);  
    Fname = aReader.GetString(2);  
    hour = aReader.GetDouble(3);  
}  
  
//close the reader  
aReader.Close();
```

aReader1.GetInt32(0) retrieves data from the course table that the query has asked for...the reader accessing the 1st column of the Course table

```
//Reading Course ID from Course table
while (aReader1.Read())
{
    //adds course number to second combo box
    comboBox2.Items.Add(aReader1.GetInt32(0));
}

//close reader
aReader1.Close();

//close the connection
aConnection.Close();
}
```

If there are any errors when trying to write or read things from the database the catch will handle the exception that is thrown

```
//Some usual exception handling
catch (OleDbException ex)
{
    Console.WriteLine("Error: {0}", ex.Errors[0].Message);
}
```

/*Connecting to database to access schedule table
and also

* reads in rows from schedule table into the
corresponding

* array that is associated with each room*/


```
OleDbCommand aCommand2 = new OleDbCommand("select * from  
Schedule", aConnection);
```

```
try  
{  
    aConnection.Open();  
  
    //create the datareader object to connect to table  
    OleDbDataReader aReader3 = aCommand2.ExecuteReader();  
  
    //Iterate through the database Schedule table  
    while (aReader3.Read())  
    {
```

The following code gets the room information from the schedule table so now the program knows what Array to put it in.

```
int room = aReader3.GetInt32(6);  
// decides which array to put it in by testing the  
//room number.  
if (room == 203)  
{  
    //creating object for tab that is associated  
with room 203
```

This is the constructor that sets all the attributes for the graphics object it reads straight from the database and into the object

```
schtrckrArray[countClass] = new  
scheduletrckr(aReader3.GetString(2), aReader3.GetString(1),  
aReader3.GetInt32(0), aReader3.GetInt32(3),  
aReader3.GetInt32(4), aReader3.GetInt32(5),  
aReader3.GetInt32(6));
```

```
countClass++;
```

```
}
```

```
else if (room == 207)
```

```
{
```

```
//creating object for tab that is associated  
with room 207
```

```
schtrckrArray2[countClass2] = new  
scheduletrckr(aReader3.GetString(2), aReader3.GetString(1),  
aReader3.GetInt32(0), aReader3.GetInt32(3),  
aReader3.GetInt32(4), aReader3.GetInt32(5),  
aReader3.GetInt32(6));
```

```
countClass2++;
```

```
}
```

```
else
```

```
{
```

```
//creating object for tab that is associated  
with room 208
```

```
schtrckrArray3[countClass3] = new  
scheduletrckr(aReader3.GetString(2), aReader3.GetString(1),  
aReader3.GetInt32(0),  
aReader3.GetInt32(3), aReader3.GetInt32(4), aReader3.GetInt32(5),  
aReader3.GetInt32(6));
```

```

        countClass3++;
    }
}

//close the reader
aReader3.Close();
}

```

```

catch (OleDbException ex)
{

    aConnection.Close();

    Console.WriteLine("Error: {0}", ex.Errors[0].Message);
}

```

```

private void tabPage1_MouseDown(object sender, MouseEventArgs e)
{
    //calculating start time
    Dhour = (int)((e.Y - 60) / 20) + 1;

    //calculating day of week
    Datef = (int)((e.X - 50) / 100) + 1;

    //calculates points for upper-left corner of highlight
    hx = ((int)((e.X - 50) / 100)) * 100 + 50;
    hy = ((int)((e.Y - 60) / 20)) * 20 + 60;

    //boolean that notifies when mouse is down
    mouse_down = true;
}

```

The major concepts in the mouse down event was to capture the coordinates and to change them into something that was a little easier to handle. Dhour, which is the start time of the class, was represented by numbers. i.e. 8:00 = 1, 8:30 = 2, 9:00 = 3
 First had to make the calculations work. So what I did was subtracted Y by 60 since this is where the grid starts. Then divided by 20, which is the width of the one rectangle on the grid. Then added 1. These gave me a way to know where I was at on the grid as far as time.
 Datef was the day of the week which was also represented by numbers. 1 = Monday 2 = Tuesday and so on. To get this number I took the x since the days of the week are driven by x and subtracted 50 which is where the x started on the grid and divide by 100 because it is 100 points away from the next x.

```

private void tabPage1_MouseMove(object sender, MouseEventArgs e)
{
    //x & y coordinates for mouse move event
    x = e.X;
    y = e.Y;
    //When mouse down is true, graphics will fill the specified
area
    if (mouse_down)
    {
        //fills specified area on tab for room 203
        using (Graphics g = tabPage1.CreateGraphics())
        {
            if ((x >= 50) && (y >= 60))
                g.FillRectangle(Brushes.LightGoldenrodYellow,
((int)((x - 50) / 100)) * 100 + 50, ((int)((y - 60) / 20)) * 20 + 60,
100, 20);
        }
    }
}

```

The Major concept with the Mouse Move event was to draw (highlight) the region that the mouse was dragging over. The g.fillRect fills the region that the mouse is moving over. The drawing element is only fired when the mouse down boolean variable is true. The mouse down boolean variable is true when the mouse down event is pressed down.

```
private void tabPage1_MouseUp(object sender, MouseEventArgs e)
{
    //calculates end time
    UpHour = (int)((e.Y - 60) / 20) + 1;
    //calculates right hand corner of object
    bttmx = ((int)((e.X - 50) / 100)) * 100 + 50 + 100;
    bttmy = ((int)((e.Y - 60) / 20)) * 20 + 60 + 20;
    //boolean that is false when mouse up event is fired
    mouse_down = false;
}
```

The major concept of the mouse up event was to capture the x & y coordinates. The coordinates calculated and the result is the bottom right corner of the graphics object. The Mouse up button also controls the bool mouse down variable. When the mouse button is released it fires and execution of the mouse up event.

```
private void button1_Click(object sender, EventArgs e)
{
    //validates professor
    if (comboBox1.SelectedItem == null)
    {
        MessageBox.Show("Please Choose a Professor", "Professor
Blank");
    }
}
```

```

    }
    else if (comboBox2.SelectedItem == null) //validates course
    {
        MessageBox.Show("Please Choose a Course", "Course
Blank");
    }
    else if ((hx < 50) && (hy < 60)) //validates that the
highlight has taken place
    {
        MessageBox.Show("Please Choose a Time Slot", "Time Slot
Blank");
    }
    else //if professor, course, and timeslot is valid
    {
        //retrieves information from combo boxes
        String Pname = comboBox1.SelectedItem.ToString();
        int CourseNum = (int)comboBox2.SelectedItem;
        //boolean to check if course conflicts
        bool check;
        //retrieves name of course from database
        String CourseNam = null;
        //prepares connection to database and query to access
table
        OleDbConnection aConnection1 = new
OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;Data
Source=srproj.mdb");
        OleDbCommand getCourse1 = new OleDbCommand("Select *
From Course Where CourseID = ?", aConnection1);
        getCourse1.Parameters.Add(new
OleDbParameter("CourseID", CourseNum));

        try
        {
            aConnection1.Open();
            OleDbDataReader crseReader1 =
getCourse1.ExecuteReader();
            while (crseReader1.Read())
            {
                //retrieves name of course
                CourseNam = crseReader1.GetString(1);
            }
            crseReader1.Close();
        }
        catch (OleDbException ex)
        {
            Console.WriteLine("Error: {0}",
ex.Errors[0].Message);
        }
        finally
        {
            aConnection1.Close();
        }

        //*****Checking if professor has any conflicts
in schedule*****//
    }
}

```

```

        if (tabControl1.SelectedTab == tabPage2)
        {
            check = checkSchedule(schtrckrArray, Pname,
Datef,Dhour , countClass,CourseNam) ||
            checkSchedule(schtrckrArray3, Pname, Datef,
Dhour, countClass3,CourseNam);

        }
        else if (tabControl1.SelectedTab == tabPage3)
        {
            check = checkSchedule(schtrckrArray, Pname, Datef,
Dhour, countClass,CourseNam) ||
            checkSchedule(schtrckrArray2, Pname, Datef,
Dhour, countClass2,CourseNam);
        }
        else
        {
            check = checkSchedule(schtrckrArray2, Pname, Datef,
Dhour, countClass2,CourseNam) ||
            checkSchedule(schtrckrArray3, Pname, Datef,
Dhour, countClass3,CourseNam);
        }
        //checks to see if course has any conflicts with other
courses
        if (check)
        {
            //if there is a conflict redraws objects on tabs
            displaySchedule();
        }
        else///pass all the checks good to process rest of data
        {
            //calculate length of course
            double Amtoftime = (((double)Uphour - (double)Dhour)/2
+ .5);

            //variables for assigned hours and hours allowed
            double profAsgnedHrs = 0;
            double profAmtAllowed = 0;
            //connect to database to retrieve information from
professor table
            //for assigned hours and hours allowed
            OleDbConnection aConnection = new
OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;Data
Source=srproj.mdb");
            OleDbCommand getProf = new OleDbCommand("SELECT * FROM
Professor Where LastName = ?", aConnection);
            getProf.Parameters.Add(new OleDbParameter("LastName",
Pname));

            //connecting to database
            try
            {
                aConnection.Open();
                //reads data from table
                OleDbDataReader getProfReader =
getProf.ExecuteReader();
                while (getProfReader.Read())
                {

```

```

        //retrieve hours for assigned and allowed for
professors
        profAsgndHrs = getProfReader.GetDouble(4);
        profAmtAllowed = getProfReader.GetDouble(3);
    }
    getProfReader.Close();
}
catch (OleDbException ex)
{
    Console.WriteLine("Error: {0}",
ex.Errors[0].Message);
}
finally
{
    aConnection.Close();
}
//variables for assigned hours and allowed hours for
course
    //and get course name from table
    double crseAsgndHrs = 0;
    double crseAmtAllowed = 0;
    String CourseName = null;
    //handles connection for database
    OleDbCommand getCourse = new OleDbCommand("Select *
From Course Where CourseID = ?", aConnection);
    getCourse.Parameters.Add(new OleDbParameter("CourseID",
CourseNum));
    //connection to database
    try
    {
        aConnection.Open();
        //reads from course table
        OleDbDataReader crseReader =
getCourse.ExecuteReader();
        while (crseReader.Read())
        {
            //retrieves information from table according to
query
            CourseName = crseReader.GetString(1);
            crseAmtAllowed = crseReader.GetDouble(3);
            crseAsgndHrs = crseReader.GetDouble(4);
        }
        //close reader
        crseReader.Close();
    }
    catch (OleDbException ex)
    {
        Console.WriteLine("Error: {0}",
ex.Errors[0].Message);
    }
    finally
    {
        aConnection.Close();
    }
    //if course hours is not over limit continue
    if (crseAmtAllowed >= (crseAsgndHrs + Amtoftime))
    {

```



```

        //adding hours to assigned hours
        crseAsgndHrs += Amtoftime;

        DialogResult result = DialogResult.Yes;
        //test if professor went over allowed hours
        if (profAmtAllowed < (profAsgndHrs + Amtoftime))
        {
            //gives choose of adding course
            result = MessageBox.Show("You have went over
Your limit. Do you still want to "+
            "add the course", "Over
Limit", MessageBoxButtons.YesNo, MessageBoxIcon.Question);

        }
        //if no then don't add course and display schedule
        if (result == DialogResult.No)
        {
            MessageBox.Show("Course was not added");
            displaySchedule();
        }
        else//if still want to add class or if professor
hours was not over limit
        {
            //add hours to professor assigned hours
            profAsgndHrs += Amtoftime;

            //adding information to corresponding array
            depending on
            //which tab(room) is currently selected
            if (tabControll.SelectedTab == tabPage2)
            {
                //adding course to array and drawing
                graphics to tabpage
                schtrckrArray2[countClass2] = new
                scheduletrckr(Pname, CourseName, CourseNum, Amtoftime, hx, hy, bttmx,
                bttmy);
                int recHeight =
                schtrckrArray2[countClass2].getheight();
                RectangleF layout = new
                RectangleF(schtrckrArray2[countClass2].downx,
                schtrckrArray2[countClass2].downy, 100, recHeight);
                Font f = new Font("Arial", 8,
                FontStyle.Bold);
                using (Graphics g =
                tabPage2.CreateGraphics())
                {
                    g.DrawString(Pname +
                    "\n" + CourseName.ToString(), f,
                    Brushes.Black, layout);
                }
                countClass2++;
            }
            else if (tabControll.SelectedTab == tabPage3)
            {

```

```

//adding course to array and drawing
graphics to tabPage
    schtrckrArray3[countClass3] = new
scheduletrckr(Pname, CourseName, CourseNum, Amtoftime, hx, hy, bttmx,
bttmy);
    int recHeight =
schtrckrArray3[countClass3].getneight();
    RectangleF layout = new
RectangleF(schtrckrArray3[countClass3].downx,
schtrckrArray3[countClass3].downy, 100, recHeight);
    Font f = new Font("Arial", 8,
FontStyle.Bold);
    using (Graphics g =
tabPage3.CreateGraphics())
    {
        g.DrawString(Pname+"\n" +
CourseName.ToString(), f, Brushes.Black, layout);
    }
    countClass3++;
}
else
{
    //adding course to array and drawing
graphics to tabPage
    schtrckrArray[countClass] = new
scheduletrckr(Pname, CourseName, CourseNum, Amtoftime, hx, hy, bttmx,
bttmy);
    int recHeight =
schtrckrArray[countClass].gettheight();
    RectangleF layout = new
RectangleF(schtrckrArray[countClass].downx,
schtrckrArray[countClass].downy, 100, recHeight);
    Font f = new Font("Arial", 8,
FontStyle.Bold);
    StringFormat format = new StringFormat();
    format.Alignment = StringAlignment.Center;
    using (Graphics g =
tabPage1.CreateGraphics())
    {
        g.DrawString(Pname + "\n" + CourseName,
f, Brushes.Black, layout, format);
    }
    countClass++;
}

//query that updates the course table; updates
the assigned hours
//for the course that was choosen
String s = "UPDATE Course SET Assignedhours = ?
WHERE CourseID = ? ";
//query that updates professor table. Updates
the assigned
//hours for professor choosen

```

```

        String s2 = "UPDATE Professor SET AssignedHours
= ? WHERE LastName = ? ";
        //Update the Schedule table. Inserts all data
into schedule table with data
        //going into corresponding columns
        String s3 = "INSERT INTO Schedule (CourseID,
CourseName, ProfessorID, StartTime, EndTime, Dayf, Room) " +
        "VALUES (@CourseID, @CourseName,
@ProfessorID, @StartTime, @EndTime, @Dayf, @Room)";

        OleDbCommand updateProfessor = new
OleDbCommand(s2, aConnection);
        updateProfessor.Parameters.Add(new
OleDbParameter("AssignedHours", profAsgnedHrs.ToString()));
        updateProfessor.Parameters.Add(new
OleDbParameter("LastName", Pname));

        OleDbCommand updateCourse = new OleDbCommand(s,
aConnection);
        updateCourse.Parameters.Add(new
OleDbParameter("Assignedhours", crseAsgnedHrs.ToString()));
        updateCourse.Parameters.Add(new
OleDbParameter("CourseID", CourseNum));
        //decides room by indication of tab selected
        String room;
        if (tabControl1.SelectedTab == tabPage2)
        {
            room = "207";
        }
        else if (tabControl1.SelectedTab == tabPage3)
        {
            room = "208";
        }
        else
        {
            room = "203";
        }

        //Adds parameters to queries
        OleDbCommand updateSchedule = new
OleDbCommand(s3, aConnection);
        updateSchedule.Parameters.Add(new
OleDbParameter("@CourseID", CourseNum));
        updateSchedule.Parameters.Add(new
OleDbParameter("@CourseName", CourseName));
        updateSchedule.Parameters.Add(new
OleDbParameter("@ProfessorID", Pname));
        updateSchedule.Parameters.Add(new
OleDbParameter("@StartTime", Dhour.ToString()));
        updateSchedule.Parameters.Add(new
OleDbParameter("@EndTime", Uphour.ToString()));
        updateSchedule.Parameters.Add(new
OleDbParameter("@Dayf", Datef.ToString()));
        updateSchedule.Parameters.Add(new
OleDbParameter("@Room", room.ToString()));

```

```

        //Connection is open and queries are ran to
update the course, professor,
        //and schedule table
        try
        {
            aConnection.Open();
            updateCourse.ExecuteNonQuery();
            updateProfessor.ExecuteNonQuery();
            updateSchedule.ExecuteNonQuery();
            aConnection.Close();
        }
        catch (OleDbException ex)
        {
            MessageBox.Show(ex.Message, "Error(s)
Committing Changes");
        }
    }
    else/// if course has to much hours
    {
        MessageBox.Show("The course selected has exceeded
it's hours", "Exceed Hours");
        //display objects on to currently selected tab
        displaySchedule();
    }
}
//reseting values
hx = 0;
hy = 0;
Datef = 0;
Dhour = 0;
Uphour = 0;
}

```

The add button is one of if not the biggest function in the whole program. When the user clicks add button the combo boxes are checked and so is the interace. If the combo boxes were empty and there was not a time or day specified (Hilight) in the form the the tabpage selected is redrawn and the the user is back under control of the project

Then the program checks to make sure there are no conflicts on the teachers schedule. If there is no conflict the program then gets data from database to test to see if there are enough course hours left and to see if the professor has went over the limit of his hours. If the

course has exceed its hours then the page is redrawn ant the control is sent back to the user. If the course hours is okay the professor hours gets tested. If the professor is ver his limits then his is prompted with a question on whether he want to continue. If not then the page is redrawn and control is sent back to the user. If the professor chooses to go on then the course is add to the array to which ever is associated to the currently selected tab. Then the information is written to the schedule table in the database.

```
private void tabPage2_MouseDoubleClick(object sender, MouseEventArgs e)
{
    //retrieving x & y for mouse double click event
    int count = 0;
    int xdc = e.X;
    int ydc = e.Y;

    //if the are objects in the array it searches for the
    object that the
    //user clicked on
    while (count < 20)
    {
        if (schtrckrArray2[count] == null)
            count++;
        else
        {
```

```

        if (xdc > schtrckrArray2[count].downx && xdc <
schtrckrArray2[count].upxc &&
            ydc > schtrckrArray2[count].downy && ydc <
schtrckrArray2[count].upyc)
        {
            countDelete = count;
            using (Graphics g = tabPage2.CreateGraphics())
            {
                int recHeight =
schtrckrArray2[countDelete].getheight();

                RectangleF layout = new
RectangleF(schtrckrArray2[countDelete].downx,
schtrckrArray2[countDelete].downy, 100, recHeight);

                g.FillRectangle(Brushes.Firebrick,
schtrckrArray2[countDelete].downx, schtrckrArray2[countDelete].downy,
100, recHeight);

                Font f = new Font("Arial", 8,
FontStyle.Bold);

                StringFormat format = new StringFormat();
                format.Alignment = StringAlignment.Center;

                g.DrawString(schtrckrArray2[countDelete].Prof+"\n" +
schtrckrArray2[countDelete].courseName, f, Brushes.Black,
layout, format);
            }

            break;

```

```

        else
        {
            count++;
        }
    }
}
}

```

The mouse double click button major concept is to locate an object on the user interface. When the mouse is double clicked the method searches for an object by using the coordinates that were captured when the user double clicked on the desired tabpage. If the object is found it is then highlighted in red and is now ready to be deleted.

```
private void button2_Click(object sender, EventArgs e)
```

Button 2 is the delete button. This button deletes an object that is selected by the mouse double click button. If the object is valid then the program proceeds to delete the object out of the corresponding array. The program then goes and deletes the information out of the schedule table in the database. It also updates the course assigned hours and updates the professor assigned hours.

```
private void button3_Click(object sender, EventArgs e)
```

Button 3 is the clear button. The clear button deletes everything from the database schedule table and also clears all the objects out of all the arrays.

```
private void button4_Click(object sender, EventArgs e)
```

Button 4 is the undo button. When the user clicks this button it refreshes and redraws the graphics on to the tab that is selected.

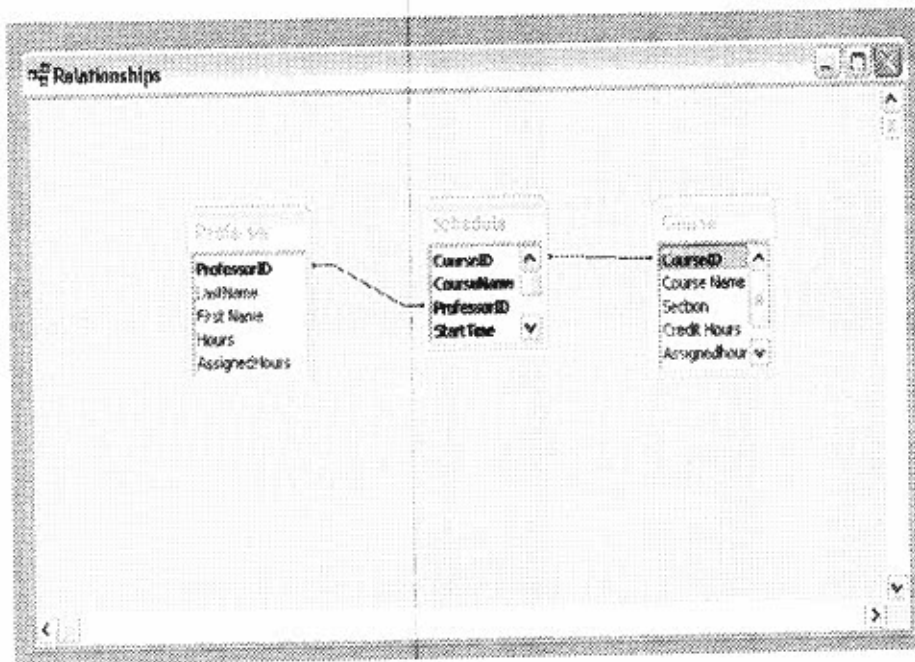
```
public void displaySchedule()
```

The displaySchedule method displays the graphics objects to a tabpage depending on the tabpage that is currently selected.

```
public bool checkSchedule(scheduletrckr[] sArray, String Prof, int Day,  
int startTime, int countC, String CourseNam)
```

The checkSchedule function checks to make sure that a professor does not have a conflict in his/her schedule. The method reads in the array which will be one of the rooms. And the program checks all the times for the current professor. If there is a conflict the user is notified. If not the program will run through the add info button code since these is where the check for conflicts happen.

Database Design



My Database was designed with 3 tables. I had a Professor table, Schedule table, Course table. The Professor table held all the information about the professors as far as last name and assigned

hours. The Schedule table held all the information about courses. The Schedule table had additional information about when a certain course would start and when it would end. It also had what day the course was scheduled. My database design probably wasn't the best design but it was good for what I wanted to do with the program. The Schedule table had foreign keys from the professor table and the course table. I did not enforce the relationships because when I tried to update the schedule table an error would come back saying I need to update the course and the Professor table.